AVBGraph: Accumulated Version Blocks for Ingestion and Analytics in Transactional Graph Storage

Sitan Ma
Peking University
Beijing, China
masitan@pku.edu.cn

Lei Zou
Peking University
Beijing, China
zoulei@pku.edu.cn

Yinnian Lin
Peking University
Beijing, China
linyinnian@pku.edu.cn

ABSTRACT

Transactional graph storage must sustain high-rate updates while serving traversal-heavy analytics over consistent snapshots. Existing designs usually favor either write throughput, using log-structured updates and versioned records that hurt scan locality, or scan efficiency, using tightly organized adjacency layouts that are costly to maintain under concurrent updates. Conventional MVCC further amplifies read overhead when long-running analytics pin multiple snapshots, forcing scans to inspect many invisible versions. We present **AVBGraph**, an in-memory graph storage engine built around AVB (Accumulated Version Blocks). AVBGraph accumulates historical updates into contiguous Version Blocks organized by invalidation epochs, replacing fine-grained version traversal with block-level pruning and sequential scans. AVBGraph combines a write-friendly update frontier with a scan-friendly historical layout, and uses epoch-based batch commit with asynchronous visibility publication to reduce reader-writer interference. We implement AVBGraph and evaluate it on graphs up to 1B edges under transactional and analytical workloads. AVBGraph achieves up to **10.6** million edge updates per second (**2.08**× over GTX), while keeping analytical performance within **1.2**× of static CSR baselines on average and substantially outperforming prior dynamic graph stores under concurrent multi-task workloads.

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Mustingu/AVBGraph.

1 INTRODUCTION

Graph data management has emerged as a cornerstone of modern data analytics. Beyond static analysis, critical systems in fields such as finance [7, 20, 22], social media [17, 34, 38], and cybersecurity [11] now operate on rapidly evolving graphs. These applications necessitate storage engines capable of handling continuous, high-velocity updates while simultaneously serving complex queries [5, 33]. Representative examples include real-time recommendation pipelines like Twitter's GraphJet [17, 34] and financial monitoring workloads that require atomic multi-edge updates to prevent logical corruption [41, 46]. Beyond single-stream processing, modern platforms increasingly support multi-tenancy [18, 30]: concurrent users execute heterogeneous analytical tasks over the same evolving graph, ranging from community detection [12] to shortest paths [6]. This pressure is further amplified by temporal learning workloads, such as dynamic GNNs, which often execute analytics over multiple historical snapshots to capture graph evolution [23, 31, 40].

We refer to such systems as Transactional Graph Storage (TGS). A TGS engine must therefore provide high-throughput ingestion, efficient analytical scans, and concurrency at the same time [2, 45]. Existing designs optimize part of this space. Write-oriented systems append updates to logs, deltas, or versioned records [46, 47], which reduces update cost but fragments adjacency access and weakens scan locality. Analytical scans then suffer from disordered neighbor access, indirection, and poor cache behavior compared with compact layouts like CSR [32]. In contrast, scan-oriented systems maintain read-friendly adjacency structures, such as sorted neighbor lists or compact dynamic indexes [8, 16]. These layouts improve read locality, but preserving them under concurrent updates introduces structural maintenance and synchronization overhead.

Multi-version concurrency further complicates this trade-off. MVCC allows long-running analytics to observe consistent snapshots while updates continue concurrently [27]. However, TGS workloads are often dominated by heterogeneous analytical tasks rather than short point queries. Different tasks may start at different times and pin different snapshots, forcing the system to retain multiple historical states simultaneously. In conventional versioned layouts, scans must repeatedly resolve visibility by inspecting obsolete or invisible records, which increases pointer chasing, branch mispredictions, and cache misses. This overhead is most severe under exactly the target setting of this paper: concurrent traversal-heavy analytics over an actively updated graph.

Our Approach: AVBGraph. We propose AVBGraph, an in-memory graph storage engine built upon AVB (Accumulated Version Blocks). AVBGraph reorganizes historical updates into Version Blocks, thereby decoupling physical data organization from logical version visibility. Instead of resolving visibility through version chains, AVBGraph accumulates historical updates into contiguous *Version Blocks (VBs)* organized by invalidation timestamp. A snapshot scan can first prune candidate blocks, and then process records sequentially within each block, improving locality and enabling batch garbage collection.

AVBGraph decouples update absorption from visibility publication. Recent updates are first loaded into a write-optimized frontier, while a background process publishes visibility in batches. This epoch-based workflow reduces coordination on the write path and minimizes disruption to long-running readers. Short-lived locks guarantee physical consistency, whereas epoch watermarks ensure logical isolation; dual-grained visibility is additionally adopted for transaction-level snapshot isolation when necessary.

Contributions. We make the following contributions:

- We propose *AVBGraph*, a transactional graph storage engine built around AVB, a block-contiguous historical-version organization that improves memory locality for analytical scans and eliminates the pointer-chasing bottleneck of conventional MVCC chains.
- We design an *epoch-based batch commit and asynchronous visibility* protocol clarified by a commit-then-publish workflow, which

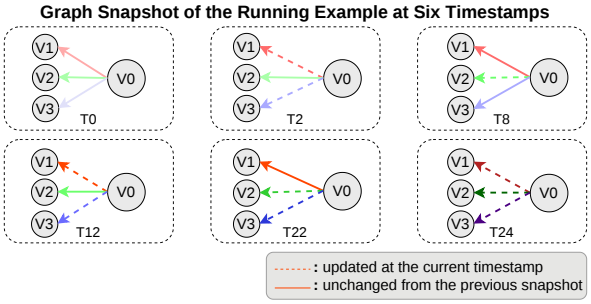

Figure 1: Graph Snapshots across six timestamps.

supports high ingestion throughput while bounding freshness latency in TGS.

- We implement AVBGraph and conduct experiments on large graphs (up to 1B edges), demonstrating up to 10.6 MEPS write throughput (2.08× over GTX) while keeping analytical slowdown close to CSR baselines (within about 1.2×), outperforming prior dynamic graph systems under concurrent multi-task workloads.

The remainder of this paper is organized as follows. Section 2 reviews the preliminaries. Section 3, 4, and 5 detail the storage and concurrency control of AVBGraph. Section 6 presents the cost model. Section 7 presents the experimental evaluation. Section 8 and Section 9 discuss related work and conclude. The full version [1] further provides appendices with supplementary technical details.

2 PRELIMINARIES AND MOTIVATION

2.1 Graph Model and Adjacency Representations

We model a directed, evolving graph as $G = (V, E)$, where an edge $e = (u, v) \in E$ connects a source vertex u to a destination vertex v; undirected graphs are represented by inserting both directions. For a vertex u, we denote its outgoing neighbor set by $N(u)$ and its out-degree by $d(u) = |N(u)|$.

The graph evolves through a sequence of snapshots $\{G_0, \ldots, G_t\}$, where each snapshot is obtained by applying an update batch Δ_t to the previous one, i.e., $G_t = G_{t-1} \oplus \Delta_t$. Consistent with prior transactional graph storage systems, we focus on edge updates. [1] Each edge carries a value field, which can also be used as a handle to external property records in property-graph settings.

To make the discussion concrete, we use a running example with six timestamps (Figure 1) throughout the paper. Consider a source vertex v_0 with three outgoing edges$e_1 = \overline{v_0v_1}, e_2 = \overline{v_0v_2}, e_3 = \overline{v_0v_3}$. The edge whose weight is updated is shown as a dashed edge.

Graph analytics are dominated by traversal access to outgoing adjacency [16]. We highlight two common access patterns: (i) *unordered adjacency scans*, which iterate over all neighbors of vertex v without order (e.g., BFS [3] and PageRank [28]), and (ii) *ordered adjacency access*, which processes neighbors in a deterministic order (typically by destination ID, e.g., triangle counting [39]).

Static layouts such as *Compressed Sparse Row (CSR)* [32] store edges contiguously with per-vertex offsets, providing excellent scan locality but poor update efficiency due to data movement.

[1]For clarity, we focus on *historical edge versions*. Historical vertex versions can be supported similarly by storing vertex updates as versioned records.

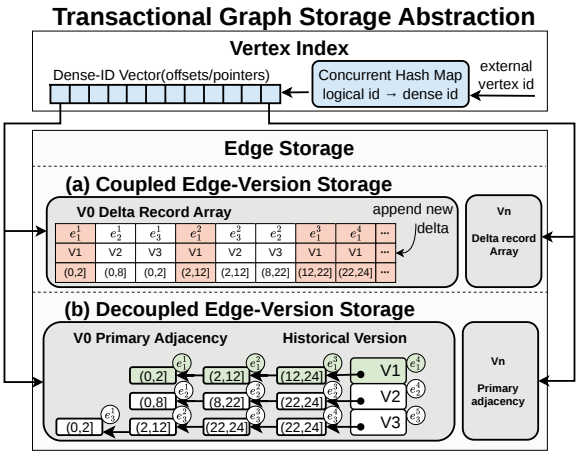

Figure 2: TGS abstraction.

Dynamic systems adopt per-vertex adjacency-list variants to localize inserts/deletes, but this introduces fragmentation during scans. Under multi-version concurrency control (Section 2.2), this cost is amplified because scans must additionally resolve version visibility, effectively adding a temporal dimension to adjacency traversal.

2.2 Concurrency Control and MVCC

To support concurrent updates and long-running analytics without blocking, TGS commonly provides Snapshot Isolation (SI) via Multi-Version Concurrency Control (MVCC). Under MVCC, an update never overwrites an edge in place; instead, it creates a new committed version, allowing readers to traverse a stable snapshot while writers continue to ingest updates. In most MVCC-based graph stores, the versioned unit is an edge. An update creates a new logical edge version, whose physical representation is a version record.

Definition 1 (Version Record and Visibility). *A version record τ is the physical representation of a logical edge version. It stores the edge payload, including the destination vertex identifier and edge value, together with a validity interval $(c(\tau), \mathrm{inv}(\tau)]$. Here, $c(\tau)$ is the creation timestamp of τ, and $\mathrm{inv}(\tau)$ is the time when τ is superseded. A record τ is visible to a snapshot at time r iff $c(\tau) < r \le \mathrm{inv}(\tau)$. The newest committed record has $\mathrm{inv}(\tau) = +\infty$. When no ambiguity arises, we use "record" as shorthand for version record.*

The invalidation time of an old record equals the creation time of its temporal successor. Therefore, the validity intervals of an edge form a continuous timeline, ensuring that exactly one committed record is visible for any snapshot time.

Example 1. *Consider edge e_1 which has three records shown in Figure 2(b): $e_1^1(0, 2]$, $e_1^2(2, 12]$, and $e_1^3(12, 24]$. The latest committed state e_1^4 is valid from 24 to $+\infty$. For a snapshot query at $r = 10$, only e_1^2 is visible.*

2.3 Transactional Graph Storage Abstraction

To enable efficient traversals and physically support the MVCC mechanisms, most TGS can be abstracted by two components: a *vertex index* and an *edge storage*, as shown in Figure 2.

2.3.1 ***Vertex Index.*** The vertex index maps sparse logical vertex IDs to dense internal IDs and locates per-vertex adjacency regions. This component is usually implemented with a concurrent hash map paired with a concurrent vector, as in Sortledton [16] and GTX [46]; AVBGraph follows the same common pattern.

2.3.2 ***Edge Storage.*** As the major structure for traversal access, edge storage largely determines scan locality and update scalability in TGS. Accordingly, several systems [8, 16, 46, 47] improve performance mainly by redesigning edge storage, albeit with different ingestion–scan trade-offs.

In general, it consists of two logical parts: *primary adjacency*, which stores the current outgoing edges of a vertex, and *historical version*, which stores older version records introduced by MVCC. Depending on whether these two parts share the same physical containers, existing systems can be classified into *coupled* and *decoupled* designs, as illustrated in Figure 2.

I. Coupled edge-version storage (e.g., LiveGraph [47], GTX [46]): These systems store adjacency as delta records, where current edge states and historical updates share the same physical containers. As illustrated in Figure 2(a), deltas of different edges and timestamps are appended into a common space. This append-dominant layout is update-friendly, but it also makes adjacency ordering impossible and forces visibility recovery to filter all unrelated or obsolete deltas. In LiveGraph, a single-edge lookup may scan the delta array to reconstruct the visible state; GTX mitigates this lookup cost with hash-chain indexing, but the coupled layout still mixes current and historical records, inflating memory footprint, scan bandwidth, and reclamation pressure.

II. Decoupled edge-version storage (e.g., Teseo [8], Sortledton [16], RapidStore [19]): Decoupling keeps the primary adjacency compact and scan-friendly, but snapshot reads must resolve visibility in an external history that is typically organized as New-to-Old (N2O) chains. Under high churn and heterogeneous long-running analytics, a reader often probes multiple snapshot-invisible head records before reaching the visible one, incurring wasted pointer chasing. RapidStore mitigates this overhead with subgraph-level validation and publication, but this coarser granularity may increase update contention and reduce fine-grained write throughput.

System	VI	Is Coupled	EdgeStorage	
LiveGraph	Hash	Coupled	TEL	
GTX	Hash+Vector	Coupled	DeltaBlocks	
System	**VI**	**Is Coupled**	**PA**	**HV**
Teseo	ART	Decoupled	FatTree+PMA	Edge-Chain
Sortledton	Hash+Vector	Decoupled	SkipList	Edge-Chain
RapidStore	Sg-Vector	Decoupled	C-ART	Sg-Chain
AVBGraph	Hash+Vector	Decoupled	Vector	VB-Chain

Table 1: Component choices under the TGS abstraction. Abbrev.: Sg = Subgraph. VI = Vertex Index. PA = Primary Adjacency. HV = Historical Version. VB = Version Block.

Table 1 summarizes representative systems under this abstraction. This paper focuses on decoupled designs: they can keep the primary adjacency compact and scan-friendly, but expose the cost of multi-version visibility resolution in external history. We analyze this N2O probing cost in Section 2.4.

2.4 Motivating Example: The Cost of N2O Visibility Probing

We use Figure 3(a) to illustrate the extra work caused by chain-based visibility resolution on the traversal path. In an N2O-style layout, each adjacency entry stores the latest committed record and a pointer to older records. For a snapshot time r, a lookup first checks the latest record. If that record is not visible, the reader follows the historical chain from newer to older records until it finds the first version visible to r.

Example 2. *Consider snapshot r=10 in Figure 3(a). Red boxes denote records that are probed but rejected, green boxes denote the visible versions, and gray boxes denote older records skipped after a match is found. For edge e_1, the latest record $(24, \infty)$ is newer than r, so the reader follows the chain, rejects $(12, 24]$, and returns $(2, 12]$. Similarly, e_2 rejects $(22, 24]$ before returning $(8, 22]$, while e_3 rejects $(22, 24]$ and $(12, 22]$ before returning $(2, 12]$. Thus, beyond the mandatory latest-record checks, the reader performs four historical-chain probes that produce no output.*

We call such a rejected historical-chain access a *snapshot-invisible touch*. Each touch reads version metadata and follows a chain pointer, and may also incur cache misses due to fragmented placement. Although only one version per edge is returned, a traversal over many edges can pay this extra cost repeatedly. The problem becomes more severe under continuous updates, where long-running readers often lag behind the latest commits and therefore encounter too-new records near the heads of version chains. Even this tiny example already performs at least 4 snapshot-invisible touches before returning three visible versions.

This motivates the design goal of AVBGraph: to avoid fine-grained chain probing on the hot traversal path. As shown in Figure 3(b), AVBGraph organizes historical versions into block-contiguous Version Blocks (Definition 2), so visibility can be resolved through block-level pruning and sequential in-block scans rather than repeated per-edge pointer chasing.

3 AVBGRAPH: STORAGE STRUCTURE

3.1 Overview

AVBGraph is a decoupled MVCC storage for transactional graph analytics. For each vertex, the newest committed edge remains in a compact primary adjacency, while older edge versions are reorganized into Version Blocks (Definition 2). This design preserves the logical semantics of per-edge multi-version visibility, but changes how historical versions are physically organized and accessed.

The key difference from conventional decoupled designs is that AVBGraph does not store history as fine-grained per-edge N2O chains. Instead, AVBGraph accumulates historical version records into block-contiguous groups. This organization is the core of AVB: it turns history access from repeated chain probing into scans over a small number of contiguous memory regions, while still preserving per-edge version lineage when needed.

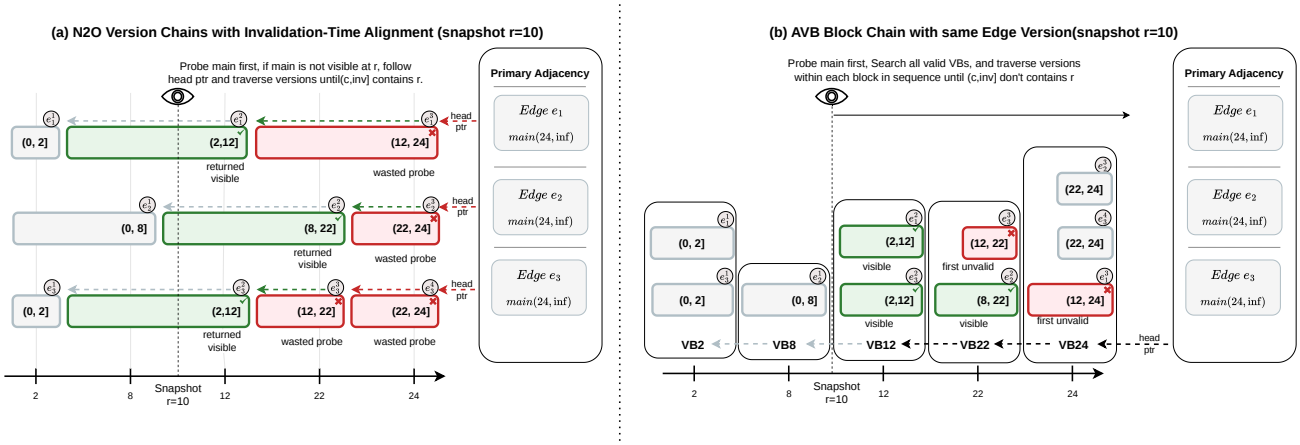

Figure 3: MVCC history access at snapshot r=10, Red, green, and gray denote wasted probes, returned visible versions, and unvisited older records, respectively. (a) N2O baseline. (b) AVB block layout.

At a high level, AVBGraph supports vertex-centric neighborhood reads and edge-granular updates: a reader accesses the neighborhood of a vertex, whereas a writer inserts, deletes, or modifies one edge at a time.

This section focuses on the layout of AVBGraph and the snapshot access path induced by that layout. We first define *Version Blocks* (Definition 2) and their per-vertex organization, then describe how snapshot reads are resolved over these blocks, and finally return to the primary adjacency as the bridge from the newest committed state to historical versions.

3.2 AVBGraph Layout

Specifically, we first define the record and block abstractions that form the basic storage unit of history, then describe how version blocks are organized per vertex while preserving version lineage across blocks, and finally revisit the running example to make the layout intuitive before moving to snapshot access.

3.2.1 Version Block. AVBGraph groups historical *version records* by invalidation timestamp. For each source vertex u, records invalidated at the same timestamp are packed into one contiguous *Version Block*. Within a block, records are ordered by creation timestamp.

Definition 2 (Version Block)**.** *For a source vertex u and an invalidation timestamp t, a* version block $VB_t(u) = \langle \tau_1, \ldots, \tau_k \rangle$ *is a contiguous sequence of historical version records of u's outgoing edges. All records in the block are invalidated at t, i.e.,* $\mathrm{inv}(\tau_i) = t$ *for every i, and are ordered by creation timestamp, i.e.,* $c(\tau_1) \le c(\tau_2) \le \cdots \le c(\tau_k)$*. The position of τ_i in the block is denoted by* $\mathrm{pos}(\tau_i) = i$*.*

Compared with a conventional N2O layout (see Figure 3(a)), AVBGraph changes the physical organization from edge-centric dispersed chains to time-grouped contiguous blocks.

Example 3 (Version Block Organization). *Figure 3(b) illustrates the VB layout for the running example. Historical records are grouped by invalidation time: for instance, $e_3^2(2, 12]$ and $e_1^2(2, 12]$ are stored in VB_{12}, while $e_2^2(8, 22]$ and $e_3^3(12, 22]$ are stored in VB_{22}. Within each VB, records are ordered by creation time; hence, in VB_{22}, e_2^2 precedes e_3^3.*

3.2.2 Per-Vertex Block Organization and Predecessor References. For each vertex, AVBGraph organizes its *version blocks* as a per-vertex block list ordered by decreasing block timestamp (e.g., Figure 3(b)). Hence, versions are ordered by invalidation time, with more recent ones appearing earlier in the list, while older versions are placed in blocks with smaller timestamps.

In MVCC-based access, each edge may require accessing multiple historical versions. In traditional edge-centric dispersed chains (e.g., Figure 3(a)), this can be done easily by following the N2O chain. However, VB changes the physical placement of versions. To preserve the logical predecessor relationship among version records, VB maintains a predecessor reference for each version within blocks. Specifically, consider a version τ. Since τ is invalidated at time $inv(\tau)$, it is stored in the block $VB_{inv(\tau)}$. Its immediate predecessor, denoted by $prev(\tau)$, is the version invalidated at the time when τ was created. Therefore, the predecessor of τ resides in the block whose timestamp equals $c(\tau)$.

We represent this relationship using a predecessor reference: $p(\tau) = (t_{\mathrm{prev}(\tau)}, \mathrm{pos}_{\mathrm{prev}(\tau)})$, where $t_{\mathrm{prev}(\tau)} = c(\tau)$ denotes the timestamp of the block containing $prev(\tau)$, and $\mathrm{pos}_{\mathrm{prev}(\tau)}$ denotes the offset of $prev(\tau)$ within $VB_{t_{\mathrm{prev}(\tau)}}$. Equivalently,

$$\tau \in VB_{inv(\tau)} \quad \Longrightarrow \quad prev(\tau) \in VB_{c(\tau)}.$$

Thus, AVBGraph preserves the per-edge temporal lineage without maintaining an explicit per-edge N2O chain. On the read path, this predecessor reference provides a logical N2O-style fallback for order-sensitive snapshot access: when the output adjacency must remain in destination order, the reader scans the ordered primary adjacency and follows predecessor references for each edge until the visible version is found.

Example 4 (Predecessor Reference). *The version $\tau = e_1^3$ is stored in VB_{24}, and its creation timestamp is* 12 *(Figure 3(b)). Thus, its predecessor resides in block VB_{12}. For e_1^3, we record its predecessor reference as $p(\tau) = (12, 2)$, which indicates that the predecessor of τ is the second record from bottom to top in VB_{12} (Figure 3(b)), i.e., e_1^2.*

At this point, the *historical version* of AVBGraph is fully defined: each vertex maintains a temporally ordered list of *version blocks*; each block groups records invalidated at the same timestamp; and

each record carries a predecessor reference. The next subsection shows that this layout naturally supports efficient snapshot reads.

3.3 Snapshot Read over AVBGraph

With the layout now established, the snapshot access path follows naturally from it. We first describe how a reader resolves visibility over the current state and the VB space, then illustrate the procedure on the running example in Figure 3(b), and finally contrast VBs with conventional N2O history to explain why the resulting read path is more locality-friendly.

3.3.1 Snapshot resolution over AVBGraph. For a snapshot read of vertex u at timestamp r, the reader first examines the newest committed adjacency state. Any current record whose validity interval contains r can be returned immediately. For the historical part, the reader traverses the per-vertex VB chain in decreasing block timestamp order. Let $VB_{t_b}(u)$ denote a version block whose records share the same invalidation timestamp t_b. The read path follows two rules:

- If $t_b < r$, the entire block is skipped.
- If $t_b \geq r$, the reader scans the block sequentially in non-decreasing order of creation timestamps and stops at the first invisible record.

The second rule follows from the organization of VB. Since all records in $VB_{t_b}(u)$ share the same invalidation timestamp t_b, once $t_b \geq r$ holds, the invalidation-side condition is already satisfied for the whole block. Visibility then depends only on the creation timestamp. Because records in a VB are stored in non-decreasing order of creation time, the first invisible record in a candidate block is exactly the first record whose creation timestamp is not smaller than r. All later records in the same block are necessarily invisible as well. Therefore, snapshot resolution over AVBGraph is performed by block-level pruning followed by a short in-block sequential scan.

Example 5 (Snapshot Read over AVBGraph). *Consider a snapshot read at $r = 10$ in Figure 3(b). Since the primary adjacency records are not visible, the reader continues into the VB chain. Blocks VB_{24}, VB_{22}, and VB_{12} are candidate blocks because their timestamps are no smaller than r, whereas VB_8 and VB_2 are skipped.*

Within VB_{24}, the first record has a creation timestamp $12 > 10$, so the scan stops without returning any record. In VB_{22}, the record $(8, 22]$ is visible, and the scan stops before the next record $(12, 22]$. In VB_{12}, records with interval $(2, 12]$ are visible and returned. This example illustrates how AVBGraph combines block-level pruning with short sequential scans inside candidate blocks.

3.3.2 Comparison with N2O. This block-oriented read path differs fundamentally from conventional edge-centric N2O chains(e.g., Figure 3(a)). In an N2O layout, snapshot resolution is edge-local and chain-centric: for each edge, the reader starts from the newest version, follows pointers one by one, and may probe several too-new versions before reaching the visible one. These probes are repeated independently across edges and are typically scattered in memory, i.e., random memory access.

In AVBGraph, by contrast, the invalidation-side test is lifted from individual records to whole blocks. Many records that would have been inspected separately in N2O are pruned together by a single block-level decision, and the remaining candidates are accessed through short sequential scans over contiguous memory. Moreover, versions from different outgoing edges that become obsolete at the same time co-reside in the same block, so visibility for multiple logical edges can be resolved within one block-oriented pass rather than through many disjoint chain traversals. AVBGraph therefore changes the granularity of history access from per-edge pointer chasing to per-vertex, block-contiguous scanning, directly reducing wasted probes and improving spatial locality for traversal-heavy snapshot reads. For example, the AVBGraph layout incurs only two wasted probes (in red) in Figure 3(b), compared to four in the conventional N2O layout in Figure 3(a). Moreover, AVBGraph enables sequential access, whereas N2O incurs random access.

3.4 Primary Adjacency

AVBGraph keeps the newest committed state of each vertex in a lightweight primary adjacency, while historical versions are stored in Version Blocks. The primary adjacency serves as the entry point for snapshot reads: a reader first checks the newest records and follows their predecessor references to the VB space when older versions are needed.

The primary adjacency consists of a *Sorted Edge Array* (SEA) and a small *Write Buffer* (WB). SEA stores stable neighbors in destination order to support sequential scans, whereas WB absorbs recent inserts and updates in an append-friendly form before being periodically sorted and merged into SEA. Each primary-adjacency record contains the destination vertex, edge value, validity metadata, and a reference to its immediate predecessor.

4 EPOCH AND DUAL-GRAINED TIMESTAMP

AVBGraph groups historical edge versions by source vertex and invalidation time. A VB therefore contains old versions of multiple edges that become invisible at the same timestamp, typically because they are replaced by the same transaction or epoch. Large update transactions naturally create many such records, so AVBGraph can amortize visibility checks, publication, and garbage collection at block granularity and scan records sequentially. For workloads dominated by small transactions, e.g., single-edge updates, each invalidation timestamp covers only a few records. The resulting VBs are smaller and provide less batching benefit. Moreover, transaction-level visibility must preserve these fine-grained commit boundaries, introducing extra metadata and coordination overhead.

To recover batching opportunities, we introduce an *epoch* abstraction that groups small write transactions within a short time window and exposes their updates at epoch granularity. AVBGraph therefore builds VBs over epochs rather than individual transactions, increasing the number of records that can share the same invalidation timestamp. When epoch-level visibility is too coarse for freshness-sensitive reads, we further refine it with intra-epoch timestamps, forming a dual-grained timestamp scheme.

4.1 Epoch: Definition and Lifecycle

We now make the epoch abstraction precise and describe how an epoch-level publication boundary is mapped to the per-vertex Version Block layout.

Definition 3 (Epoch)**.** *An epoch E_e is a write-admission interval associated with an epoch identifier e. Every write transaction admitted while E_e is open is assigned e and belongs to the same epoch-level*

publication unit. Therefore, coarse-grained readers do not distinguish the individual commit times of transactions within the same epoch; they observe the epoch only after its publication boundary becomes visible.

An epoch is a logical visibility boundary rather than a physical history container. When epoch E_e is published, the version records made historical by its updates are reorganized into per-vertex Version Blocks. For each affected vertex u, these records are stored in $VB_e(u)$, where the epoch identifier e serves as the coarse-grained invalidation timestamp, in the same sense as timestamp t in Section 3. Thus, one epoch may materialize as multiple VBs across different vertices, while for a fixed pair (e, u) there is at most one $VB_e(u)$. Epochs determine visibility boundaries, whereas VBs remain the physical units for storing and scanning historical versions.

Before publication, AVBGraph keeps records of unpublished epochs in transient per-vertex structures, called Transient Version Blocks (TVBs). A TVB is write-friendly: it is append-only and does not enforce creation-time ordering. Once all transactions in an epoch have completed, its TVB records become eligible for background publication. During publication, these records are reorganized into immutable VBs sorted by non-decreasing creation timestamp c.

Each epoch moves through four states:

- *Open*: the epoch accepts incoming write transactions.
- *Sealed*: the epoch no longer accepts new transactions, but assigned transactions may still be running.
- *Committed*: all transactions assigned to the epoch have completed, but the epoch is not yet reader-visible.
- *Published*: the epoch has been reorganized into per-vertex VBs and becomes visible to analytical readers.

AVBGraph maintains two global epoch markers:

- *gwe*: the unique epoch currently in the *Open* state;
- *gre*: the latest epoch in the *Published* state.

These markers separate write admission from read visibility. Writers are assigned to the current open epoch *gwe*, while coarse-grained readers observe only the published prefix up to *gre*.

Epoch State Transitions.

(1) *Open* → *Sealed.* When the accumulated update volume of the current open epoch reaches the target threshold N, the epoch is sealed. A new epoch is then created as the next open epoch, and *gwe* is advanced to it.
(2) *Sealed* → *Committed.* A sealed epoch becomes *Committed* after all transactions assigned to it have completed.
(3) *Committed* → *Published.* A background publication worker processes epochs in increasing order, starting from *gre* + 1. If the next epoch is committed, the worker reorganizes its TVB records into per-vertex VBs, publishes the epoch, and advances *gre*. Multiple consecutive committed epochs may be published in one pass.

Ordered publication preserves a monotone reader-visible history: even if a later epoch commits earlier, it cannot become visible before older unpublished epochs.

Example 6 (Epoch Lifecycle). *Figure 4 illustrates the lifecycle with sealing threshold $N = 2$. Initially, gwe points to E1. After Txn1 and Txn2 are admitted to E1, E1 is sealed and E2 becomes the new open epoch. Once all transactions in E1 finish, E1 becomes committed, is published by the background worker, and advances gre to E1.*

The same rule applies to later epochs. Even if E3 commits before E2, the publication worker must first publish E2 because it starts from gre + 1 and proceeds in epoch order. Only after E2 becomes visible can E3 be published. Finally, E4 is shown as the current open epoch.

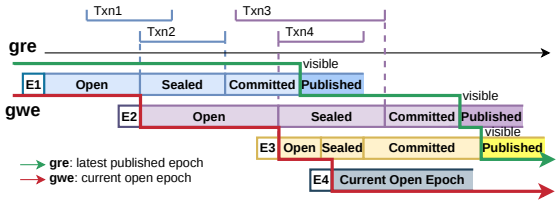

Figure 4: Example of epoch lifecycle in AVBGraph.

4.2 Low-overhead Epoch Management

The lifecycle in Section 4.1 defines the logical epoch states. AVBGraph implements this lifecycle with two lightweight mechanisms: sampled sealing on the write path and slot-based asynchronous publication. Figure 5 summarizes the workflow.

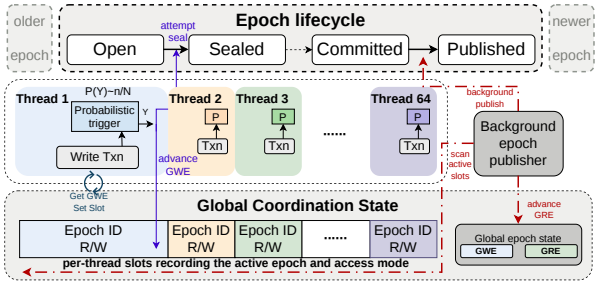

Figure 5: Low-overhead epoch management in AVBGraph.

4.2.1 Decentralized Sampled Sealing. Maintaining an exact per-epoch update counter would make every update touch the same shared state. AVBGraph avoids this bottleneck through sampled sealing. Each epoch has a small sampling quota h, where $h \ll N$. Conceptually, each update generates a sealing hit with probability $p = h/N$. A transaction with n updates draws $X_n \sim \text{Binomial}(n, p)$ and adds X_n to the epoch's sampled-hit counter. If the counter crosses h, the transaction attempts to seal the epoch. Thus, an epoch requires only h expected counter hits rather than N per-update counter increments.

Let U be the number of updates admitted before sealing. Then U follows a negative-binomial distribution with $\mathbb{E}[U] = N$ and $\text{Var}(U) = N(N-h)/h$. Therefore, the concentration can be quantified by standard Chernoff bounds. For example, when $h = 32$, the negative-binomial distribution gives a tightly concentrated epoch size: for large N, $\Pr[0.5N \le U \le 1.5N] \approx 99.4\%$. This illustrates that a small sampling quota already keeps epoch sizes close to the target while avoiding a per-update global counter.

When a transaction observes that the sampled quota is reached, it acquires a lightweight sealing lock and rechecks whether *gwe* still points to the same epoch. If so, it seals the epoch, creates a new open epoch, resets the sampled counter, and advances *gwe*; otherwise, it aborts the attempt. This lock is acquired only when sealing is triggered and therefore stays off the common write path.

4.2.2 Asynchronous Commit Detection and Ordered Publication. After an epoch is sealed, AVBGraph detects when all its transactions have finished without per-epoch counters. Each worker records its current epoch and access mode in a local activity slot. A background publication worker periodically scans these slots; when the next sealed epoch after *gre* is no longer referenced by any active writer, it marks the epoch as *Committed*, reorganises its TVB records into VBs, publishes the epoch, and advances *gre*. Publication follows epoch order, so readers see only a prefix of published epochs. To avoid missing writers that join an epoch concurrently, a worker registers with a lightweight guard: it reads *gwe*, writes that epoch into its slot, and re-reads *gwe*; if *gwe* changed, it retries.

Appendix A.2 of the full version [1] gives the full negative-binomial derivation and additional implementation details of slot-based publication.

4.3 Dual-Grained Timestamp Extension

Epoch-level visibility enables AVBGraph to batch version construction and publication at VB granularity, but it may delay the visibility of newly committed updates until the whole epoch is published. For freshness-sensitive reads, AVBGraph optionally provides a dual-grained timestamp extension: the epoch remains the physical grouping and publication unit, while an intra-epoch commit order refines visibility among committed transactions in the same epoch.

Definition 4 (Composite Timestamp). *A composite timestamp is a pair $T = (e, t)$, where e is the epoch identifier and t is the intra-epoch commit order. Composite timestamps are compared lexicographically. Under the dual-grained extension, the creation timestamp $c(\tau)$, the invalidation timestamp $inv(\tau)$, and the reader snapshot timestamp T_r are all interpreted as composite timestamps.*

For each vertex u, updates in an unpublished epoch are temporarily recorded in a transient version block $TVB_e(u)$. In the dual-grained write path, a writer holds the write locks of all touched vertices until commit, so uncommitted records are not exposed to readers on the same vertex. The detailed locking protocol is described in Section 5. At commit time, the transaction obtains an intra-epoch commit order and appends its records to the corresponding TVBs. TVBs are not sorted or reorganized at commit time; they are reorganized into immutable, creation-ordered VBs only during publication.

A dual-grained reader takes a snapshot timestamp $T_r = (e_r, t_r)$ and observes all published epochs before e_r, together with committed updates in epoch e_r whose intra-epoch commit order is at most t_r. A version record τ is visible iff $c(\tau) < T_r \leq inv(\tau)$, where comparisons are lexicographic. We call e_r the reader's active epoch.

Visibility over VBs. Let e_b be the epoch label of a published VB. Dual-grained visibility follows the same block-oriented access path as coarse-grained visibility, except when the block belongs to the reader's active epoch:

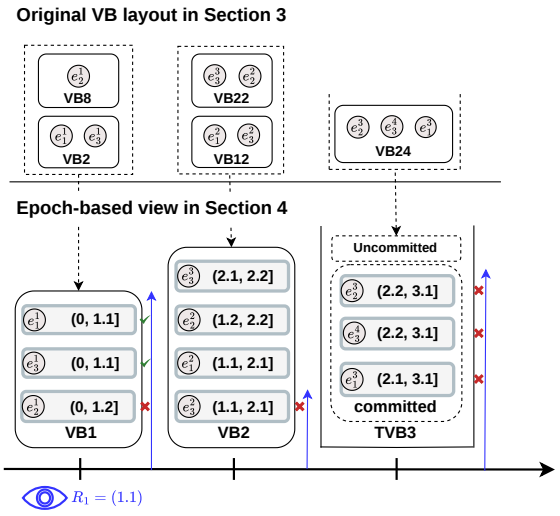

Figure 6: Epoch-based reorganization and dual-grained read paths.

- $e_b < e_r$. The block is skipped.
- $e_b > e_r$. The reader scans records in creation-timestamp order and stops once $c(\tau)$ is not earlier than T_r.
- $e_b = e_r$. The reader also scans in creation-timestamp order, but for the scanned prefix it must check the full condition $c(\tau) < T_r \leq inv(\tau)$, because records may be invalidated at different intra-epoch times.

Thus, the only extra work over published VBs appears in the active-epoch case, where epoch-level pruning alone is insufficient to determine visibility.

Visibility over TVBs. A TVB belongs to an unpublished epoch and is not sorted by composite creation timestamp, so it does not support creation-order early stopping. When a reader consults $TVB_e(u)$, it first acquires the read lock of vertex u; therefore, any observed TVB records are already committed. The reader then scans the TVB and tests each record against $c(\tau) < T_r \leq inv(\tau)$.

Example 7 (Read Paths under Dual-Grained Visibility). *Consider a reader with snapshot timestamp $R_1 = (1, 1)$. For a published VB VB_2 with $e_b > 1$, the reader performs an ordered scan and stops at the first invisible record $(1.1, 2.1]$. VB_1 corresponds to the active-epoch case, so the reader stops at the first record whose creation time is not earlier than R_1. In this example, it scans all the records. For the unpublished TVB_3, the reader scans the committed records and evaluates the full composite-timestamp condition.*

Overall, dual-grained timestamps restore transaction-level snapshot visibility while preserving epoch-based grouping, ordered publication, and block-oriented historical layout. Finer checks are needed only for the active-epoch VB case and for TVB records in the current unpublished epoch.

5 CONCURRENCY CONTROL

In this section, we propose a concurrency control strategy for the AVBGraph system to coordinate between writers, readers, and

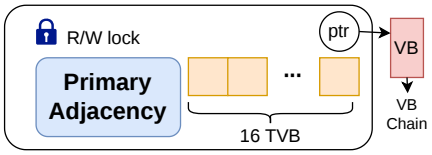

Figure 7: Lock-protected structure in AVBGraph.

the background publisher on the shared graph structures under concurrent access, while ensuring both structural correctness and snapshot-consistent access.

Note that once a VB (version block) has been published, it is no longer modified in place. The concurrency control of AVBGraph is thus focused on a per-vertex *primary adjacency state*, which encompasses the primary adjacency (i.e., the SEA and WB introduced in Section 3.4), the TVB, and the head pointer of the VB chain, as shown in Figure 7. This state is protected by a per-vertex read-write lock, ensuring that destructive updates to local adjacency and unpublished history remain physically consistent.

5.1 Concurrency Control Protocol

Shared Execution Workflow. We first introduce the parts of AVBGraph's execution workflow that are shared by both coarse-grained and dual-grained visibility. In both modes, AVBGraph uses three logical worker roles: reader, writer, and publisher.

Reader. A reader first acquires the per-vertex read lock and reads the current primary adjacency together with the head pointer of the VB chain. After these pointers are captured, the reader releases the lock and performs the remaining traversal without synchronization.

Writer. A writer acquires the per-vertex write lock and appends the update information into the temporary version buffer (TVB). For a logical MVCC update, the latest value is reflected in the primary adjacency, while the displaced old value is recorded into the TVB as a historical version. The exact way in which this logical transformation is materialized depends on the chosen visibility granularity and is described separately below.

Publisher. A publisher acquires the per-vertex write lock, transforms the accumulated TVB records into one or more VBs, links the new VBs to the head of the VB chain, and then releases the lock.

Coarse-grained mode. Since visibility is bounded by epochs, a writer does not need to make its update immediately visible to readers. It writes the new record into the TVB and can release the lock after each edge update. During publication, the publisher transforms the TVB and swaps the latest records between Primary Adjacency and TVB, so that Primary Adjacency contains the newly published state.

Dual-grained mode. Updates must become visible to readers immediately. Therefore, readers may also need to access TVB while holding the lock. A writer installs the newest record into Primary Adjacency, moves the replaced record into TVB, and keeps the lock until the transaction commits. The publisher only performs TVB transformation, without swapping records between Primary Adjacency and TVB.

In short, coarse-grained mode delays visibility to amortize publication cost, whereas dual-grained mode pays extra synchronization cost to expose intra-epoch updates immediately.

Deadlock and abort handling. AVBGraph uses a bounded no-wait locking policy. If a transaction fails to acquire a required lock, it releases the locks already acquired in the current attempt, backs off, and retries; after a bounded number of failures, it aborts.

Abort handling follows the installation point of each mode. In coarse-grained mode, updates have not been installed into the primary adjacency before publication, so abort only removes or marks the transaction's TVB records. A sealed epoch is published only after all transactions in the epoch have committed or aborted. In dual-grained mode, the writer still holds the affected locks at abort time, so it rolls back the primary adjacency using the undo information in the TVB and then removes or marks its TVB records. The publisher materializes only committed records.

5.2 Correctness, Reclamation, and Extensions

Beyond the basic locking protocol, we examine correctness guarantees, safe reclamation of obsolete history in AVBGraph, and several optional extensions. We summarize these aspects here; Appendices B–D of the full version [1] provide the detailed proofs and arguments.

Isolation and correctness. Under the protocol above, AVBGraph provides *snapshot isolation* under both visibility modes. In the dual-grained mode, AVBGraph provides transaction-level snapshot isolation at vertex-local protection granularity, which can be viewed as an MV2PL-style implementation. In the coarse-grained mode, AVBGraph provides snapshot isolation at epoch granularity, where transactions admitted into the same epoch share one reader-visible commit boundary.

Garbage collection. AVBGraph reclaims obsolete published VBs only after epoch advancement proves that they are unreachable by all active readers. Since historical versions are accumulated into contiguous VBs, reclamation is performed at block granularity rather than along per-edge version chains.

Read-write and serializable extension. Dual-grained AVBGraph supports read-write transactions by retaining read locks until commit, which yields a 2PL-style serializable variant, but it is mainly suitable for short OLTP-style workloads rather than long graph analytics with large read and write sets. Similarly, coarse-grained AVBGraph can be extended to serializable execution by holding read/write locks until commit and keeping writes in transaction-private versions before publication.

6 COST MODEL OF N2O CHAINS AND AVB

In this section, we develop a quantitative cost model to explain how AVB, the historical version layout used in AVBGraph, reduces snapshot visibility overhead compared with N2O version chains. The model is not intended to predict exact runtime, but to capture the dominant memory-access cost caused by different organizations. Consider a snapshot scan of a vertex v with out-degree d. Let Δ be the lag in epochs between the reader snapshot and the latest state, μ be the mean number of updates per edge per epoch, and c_s and c_r be the costs of one sequential access and one random or pointer-chasing access, respectively. At the block level, the adjacency of v receives updates with aggregate rate $d\mu$ per epoch.

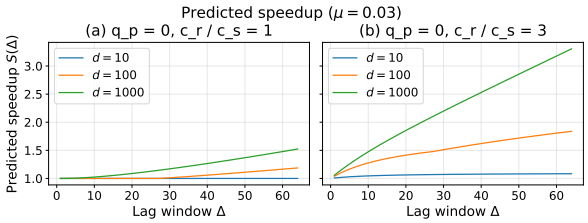

Figure 8: Predicted speedup of AVB over chain-based visibility resolution under two parameter settings.

We model the number of updates received by one edge during the lag window as Poisson($\mu\Delta$). Thus, an edge is updated at least once with probability $1 - e^{-\mu\Delta}$. At the block level, vertex v contributes a non-empty VB in an epoch with probability $1 - e^{-d\mu}$. Let d_h be the expected number of affected edges and $B_v(\Delta)$ be the expected number of non-empty candidate VBs. Then

$$d_h = d(1 - e^{-\mu\Delta}), \qquad B_v(\Delta) = \Delta(1 - e^{-d\mu}).$$

N2O Chain Cost. In an N2O layout, the reader resolves visibility independently for each edge. After scanning the current adjacency, it may follow version-chain pointers and skip newer versions that are invisible to the reader's snapshot. The expected cost is

$$C_{\text{chain}}(v, \Delta) \approx dc_s + d\mu\Delta c_r.$$

Here, dc_s is the cost of scanning the current adjacency, while $d\mu\Delta c_r$ captures the dominant overhead of pointer-chasing probes over historical versions.

AVB Cost. In AVB, historical versions are grouped into VBs. For each candidate VB, a reader pays one random block-entry cost and then scans in-block records sequentially. Let $q_p \in [0, 1]$ be the fraction of candidate VBs that are *pure*, meaning that all versions in the VB are visible to the snapshot. For a pure VB, the block-entry access already accounts for the first visible version, so charging all visible versions again as sequential accesses would overcount one touch. For a non-pure VB, the scan may also touch one invisible boundary version, which offsets this overcount. Thus, only pure VBs reduce the effective sequential work:

$$x_{\text{seq}}(\Delta) = \min\{d_h - \min(q_p B_v(\Delta), d_h), d\mu\Delta - B_v(\Delta)\}.$$

The AVB cost is therefore

$$C_{\text{AVB}}(v, \Delta) \approx dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s.$$

Here, dc_s is the current-adjacency scan cost, $B_v(\Delta)c_r$ is the random entry cost for candidate VBs, and $x_{\text{seq}}(\Delta)c_s$ is the remaining sequential in-block scan cost.

The expected speedup of AVB over N2O visibility resolution is

$$S(\Delta) = \frac{C_{\text{chain}}(v, \Delta)}{C_{\text{AVB}}(v, \Delta)} \approx \frac{dc_s + d\mu\Delta c_r}{dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s}.$$

Figure 8 illustrates the predicted speedup under two settings. The left subplot uses a conservative setting with $q_p = 0$ and $c_r/c_s = 1$, where AVB receives no pure-block credit and random access is not penalized relative to sequential access. The curves remain close to 1, showing that AVB is not predicted to be worse even under this lower-bound setting. The right subplot keeps $q_p = 0$ but sets $c_r/c_s = 3$, reflecting that an N2O probe chases a pointer to a fresh record at roughly cache-line granularity, where measurements of DDR5 and DRAM baselines report random access about 2-3× slower than sequential at 32-64 B [4, 42]. The qualitative trend is insensitive to the exact ratio: speedup grows with the lag window Δ and is more pronounced for larger d, because N2O pays many edge-level random probes while AVB amortizes visibility resolution over candidate VBs and performs the remaining work sequentially.

Remark on dual-grained visibility. With the dual-grained timestamp, a snapshot scan may additionally inspect committed records in unpublished TVBs reached by the scan. Because temporal consistency makes earlier-started transactions usually finish earlier, each unpublished epoch is typically delayed by only a few remaining transactions. Therefore, the scan touches only a small number of such TVBs in practice, and this extra cost remains a bounded second-order overhead.

Appendix E of the full version [1] derives d_h, $B_v(\Delta)$, and the refined $x_{\text{seq}}(\Delta)$ term, including the pure-VB credit and the additional accounting bounds.

7 EXPERIMENTS

Experimental Setup. All experiments were conducted on a dual-socket server with two Intel Xeon Gold 5318Y CPUs at 2.10 GHz, 48 physical cores, 96 hardware threads, 72 MiB LLC, and 1.2 TiB memory. Unless otherwise stated, we use 64 worker threads, which creates high read-write contention while leaving hardware contexts for the OS and background publication. All systems were compiled with GCC 9.4 using `-O3`.

Dataset. We evaluate AVBGraph on synthetic and real-world dynamic graphs, summarized in Table 2. The synthetic datasets include Graph500 (R-MAT) graphs with power-law degree distributions and Uniform random graphs. The real-world datasets include DOTA-League and Edit-Wiki.

Dataset	Type	$\|V\|$	$\|E\|$	Max Degree
DOTA-League	Real	61K	51M	17004
Edit-Wiki	Real	50M	572M	5576228
Graph500-24	R-MAT	9M	260M	406416
Graph500-26	R-MAT	33M	1B	1003338
Uniform-24	Uniform	9M	260M	103

Table 2: Datasets used in the evaluation.

Baselines and Isolation. We compare AVBGraph with GTX [46], LiveGraph [47], Teseo [8], Sortledton (SLT) [16], RapidStore [19], and CSR as a read-only upper-bound reference. All systems are evaluated under snapshot isolation or closest equivalent. AVBGraph has two modes: AVBGraph uses epoch-level snapshot isolation, while AVBGraph-dual provides transaction-level snapshot isolation. LiveGraph and SLT are evaluated under serializable isolation. For RapidStore, we guard reader registration with a lock and disable GC and part of version generation. Our results therefore represent an upper bound for a correct implementation, and memory usage is omitted (see Appendix F.3 in full version [1] for details).

Workload and Metrics. Our evaluation covers write-only ingestion, read-only analytics, concurrent analytics under continuous updates, and memory-footprint profiling. The analytics include PageRank (PR), breadth-first search (BFS), single-source shortest path (SSSP), weakly connected components (WCC), local clustering coefficient (LCC), and community detection label propagation

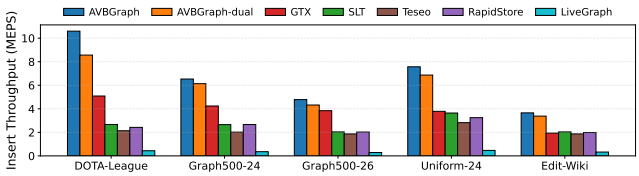

Figure 9: Edge insertion throughput with 64 threads.

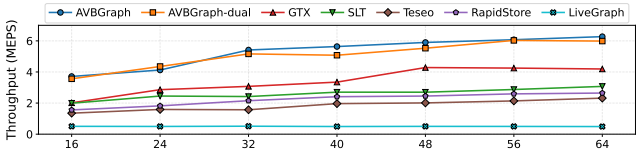

Figure 10: Insert scalability (threads from 16 to 64)

(CDLP). We report throughput, latency, and memory consumption, averaged over 5 runs unless otherwise stated.

7.1 Insert Experiment

Experiment Setup. We first evaluate pure insertion throughput under the standard checked-insertion setting: before inserting an edge, the system verifies that the edge does not already exist. All input edges are inserted in a random order. This setting avoids locality-friendly update bursts and provides a conservative measurement of the generic transactional insertion path.

Results and Analysis. Figure 9 shows that AVBGraph achieves the highest insertion throughput on all datasets, reaching 10.60 M inserts/s on DOTA-League and 6.53 M inserts/s on Graph500-24. It outperforms GTX by 2.08× and 1.38×, respectively, and remains clearly ahead of read-oriented baselines such as SLT and Teseo.

The main reason is that AVBGraph keeps the write path lightweight. New updates are absorbed by the primary adjacency and write-friendly buffers, while visibility publication and reorganization are amortized through epochs and VB construction. As a result, AVBGraph avoids the high structural maintenance cost of fully ordered dynamic adjacency structures and the scan/update metadata overhead of coupled version layouts.

AVBGraph-dual has lower throughput than coarse-grained AVBGraph because transaction-level visibility introduces additional timestamp and synchronization costs. However, the gap is moderate, and AVBGraph-dual still outperforms all other baselines across datasets. This shows that finer freshness can be supported without losing the write-side advantage of AVB's epoch-based organization.

Scalability Analysis. We evaluate the write throughput by varying the concurrent threads from 16 to 64. As shown in Figure 10, AVBGraph consistently delivers the highest throughput across all configurations, closely followed by AVBGraph-dual which exhibits similarly competitive performance. Both systems scale effectively and approach their optimal performance at 64 threads.

7.2 Static Graph Analysis

Figure 11 reports read-only analytics on stable snapshots. Since no concurrent updates are running, this experiment mainly isolates the efficiency of the current-graph layout and adjacency scans, rather than snapshot visibility resolution. BFS on DOTA-League finishes within only a few milliseconds, so small differences should not be over-interpreted.

Overall performance. AVBGraph achieves strong read-side performance across most analytics. With a geometric mean slowdown of about 1.2× relative to CSR, it significantly outperforms typical dynamic stores. This shows that AVBGraph does not sacrifice scan efficiency for update support. Its primary adjacency keeps stable neighbors in a compact ordered array and uses a lightweight write buffer for recent updates, so traversal-heavy algorithms can still benefit from mostly sequential adjacency scans.

Analysis. Coupled dynamic stores mix current edges with update or state metadata, and may still encounter obsolete entries during scans. This increases scan bandwidth and weakens locality even on stable snapshots. GTX further pays overhead from its complex lock-free structure, including additional metadata checks on the read path. Decoupled ordered stores such as SLT and Teseo maintain fully ordered dynamic structures, which can help order-sensitive workloads such as LCC, but their skip-list or PMA-based layouts introduce extra structural overhead for general analytics. In contrast, AVBGraph uses a simpler ordered-array layout plus a small write buffer, retaining most scan locality while avoiding the cost of fully dynamic ordered indexes. RapidStore also shows no clear advantage here, since its subgraph-level version organization is not more efficient than AVBGraph's compact adjacency-oriented representation for scan-heavy analytics. AVBGraph-dual follows the same trend as AVBGraph and incurs only a small slowdown from additional metadata and timestamp checks for transaction-level snapshot visibility. This indicates that finer-grained freshness preserves most of AVBGraph's read-side efficiency.

7.3 Concurrent Analytics and Updates

Setup and Workloads. We use **Graph500-24** for the concurrent evaluation to match the setting used by prior concurrent studies [8, 16, 19, 46]. After the graph is loaded into memory, update transactions and snapshot analytics run concurrently. By default, we use 64 threads: 32 threads continuously issue random edge-update transactions, while 32 threads run analytics over consistent snapshots. Since AVBGraph sustains much higher update throughput than the baselines, its readers may experience stronger write-side pressure under 32W+32R. We therefore also report 16W+32R for AVBGraph variants.

We vary the number of concurrent read tasks to emulate multi-tenant analytics over an evolving graph. The 32 reader threads are evenly divided among 1, 2, 4, or 8 tasks, which creates multiple active snapshots and stresses snapshot retention and visibility resolution. Homogeneous workloads run either PageRank or BFS for all reader tasks, while heterogeneous workloads split the tasks equally between PageRank and BFS, creating different snapshot lifetimes. Colors and markers denote systems, and line styles denote update/read settings.

Consistent with prior concurrent evaluations [16, 46], Teseo is excluded due to an unresolved concurrent bug. LiveGraph is omitted as its throughput is over an order of magnitude below other systems in this mixed setting. RapidStore is included after the fix but exhibits substantially higher conflict rates in concurrent execution.

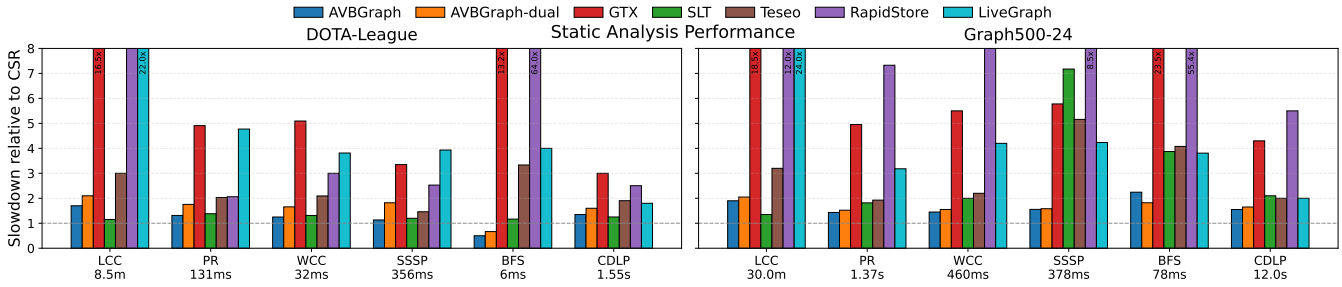

Figure 11: Static analysis performance normalized to CSR.

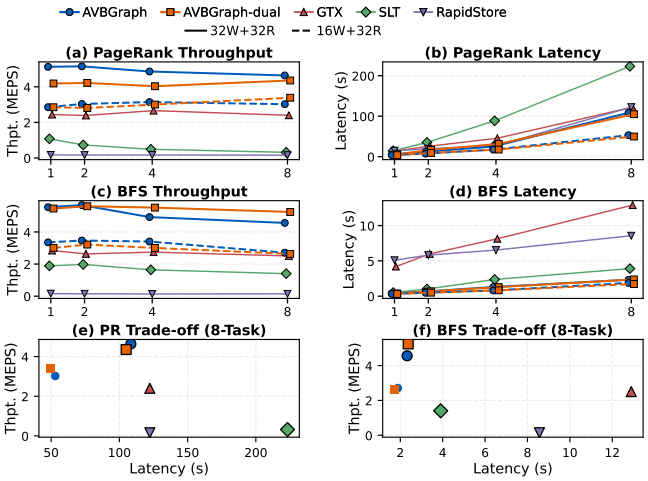

Figure 12: Performance scalability with varying read tasks (a–d) and the throughput-latency trade-off under 8 tasks (e–f). Thpt. stands for throughput in the figure.

Homogeneous analytical workloads. Figure 12 shows the homogeneous workloads, where all readers run the same algorithm. PageRank is long-running and scan-intensive, while BFS is a shorter traversal workload. Across both workloads, AVBGraph provides the strongest overall operating point: it sustains the highest update throughput while keeping analytical latency competitive. As the number of read tasks increases, latency rises because each task receives fewer reader threads and more snapshots remain active. AVBGraph degrades more gracefully because published history is organized into Version Blocks, allowing retained versions to be accessed through block pruning and sequential scans rather than repeated per-edge probing.

The baselines expose different bottlenecks under concurrent updates. Although SLT is competitive on read-only PR and BFS, its PR latency degrades much more than BFS once updates run concurrently, because long scans keep snapshots alive and repeatedly interact with protected adjacency and version state. Even for BFS, SLT falls clearly behind AVBGraph as the number of read tasks increases. GTX still suffers from read-path metadata and coupled-version layout overhead, while RapidStore has low update throughput under conflicts. In contrast, AVBGraph separates update absorption from historical-version access, preserving both high ingestion throughput and stable read latency. The 16W+32R results further confirm this trend: even when AVBGraph uses fewer writers, it still achieves higher update throughput and lower analytical latency than the baselines across the measured points. AVBGraph-dual follows the same trend with moderate overhead, showing that transaction-level visibility does not break the mixed-workload advantage.

To visually consolidate this read-write balance under peak concurrency, Figures 12e and 12f map the 8-task execution onto a throughput-latency trade-off space, where bordered and borderless markers denote `32W+32R` and `16W+32R` workloads, respectively. In these plots, the optimal performance frontier lies at the top-left corner (maximum throughput and minimal latency). Reflecting the aforementioned bottlenecks, both AVBGraph and AVBGraph-dual sit closest to this ideal frontier.

Heterogeneous analytical workloads. We next evaluate heterogeneous workloads, where PageRank and BFS run together. This setting is more demanding because PageRank tends to pin older snapshots for longer, while BFS may finish earlier and observe newer graph states.

As shown in Figure 13, AVBGraph continues to provide the best read-write balance. It sustains high update throughput and keeps both PageRank and BFS latency low across task counts. The reason is that AVBGraph resolves historical visibility over accumulated VBs, which is especially effective when different readers operate on different snapshot ages. Instead of following many edge-local version chains, readers scan compact blocks with better locality.

The heterogeneous setting further exposes the cost of multi-snapshot execution for the baselines. SLT's BFS latency degrades more than in the homogeneous BFS case, indicating that mixed snapshot lifetimes amplify the overhead of protected adjacency access and per-edge version resolution. GTX shows lower PageRank latency than AVBGraph only at the 8-task point under 32W+32R, but this comes with much lower update throughput, meaning that its readers face weaker write-side pressure. Under the 16W+32R setting, AVBGraph outperforms GTX on both update throughput and PageRank latency, and remains better than the other baselines across the measured read and write metrics. AVBGraph-dual also remains stable: its lower update pressure can reduce latency in some cases, but the key result is that finer-grained freshness is supported without a large read-write throughput collapse.

A component study under this same 32W+32R setting—isolating the epoch size N and sampled sealing—appears in the full version [1] (Appendix F): ingestion throughput rises with N and saturates as larger epochs build larger, better-amortized VBs, while removing sampled sealing reduces throughput by 11–18%. Coarse- versus dual-grained visibility (AVBGraph vs. AVBGraph-dual) reported throughout this section forms a built-in visibility-granularity ablation.

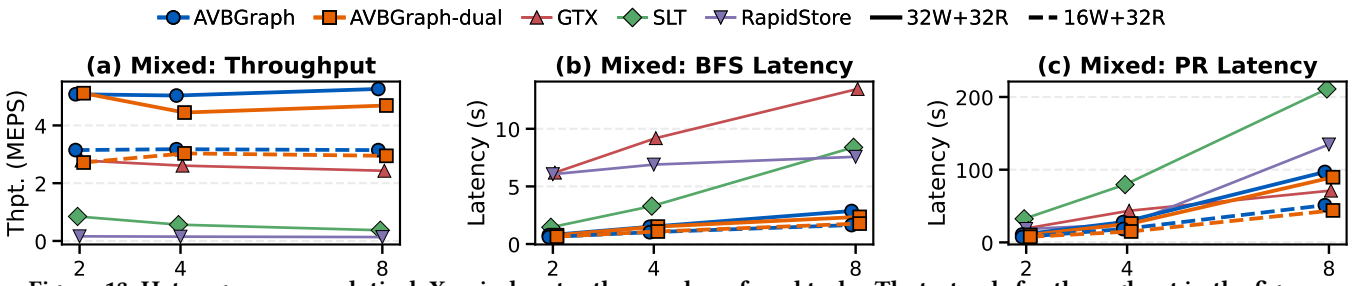

Figure 13: Heterogeneous analytical. X-axis denotes the number of read tasks. Thpt. stands for throughput in the figure.

7.4 Memory Footprint

Experimental setup. We measure process-level RSS externally and subtract the RSS at the initialized state. Since systems finish in different wall-clock times, we normalize execution progress: the first 20% corresponds to insertion, and the remaining 80% corresponds to the concurrent read-write phase. RapidStore is omitted because the stability modification makes its memory footprint incomparable.

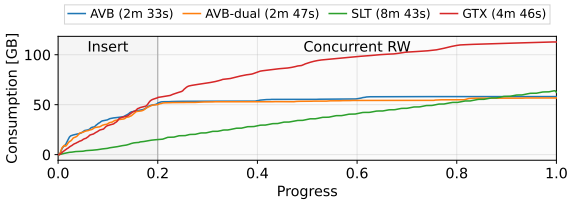

Figure 14: Memory consumption during insertion and concurrent read-write execution.

Results and analysis. Figure 14 shows that both AVBGraph modes quickly flatten after entering the concurrent read-write phase, staying in the high-50 GB range. This indicates that AVBGraph keeps retained history stable when old snapshots are pinned, because historical versions are published and reclaimed at Version Block granularity. AVBGraph-dual follows a similar curve, showing that transaction-level freshness does not introduce noticeable memory amplification.

SLT has a lower insert-phase footprint, but it takes much longer to finish the same workload: 8m 43s, compared with 2m 33s for AVBGraph and 2m 47s for AVBGraph-dual. Its memory usage also keeps growing during concurrent execution and eventually surpasses both AVBGraph variants. GTX shows the largest memory amplification, ending above 110 GB due to higher version metadata and harder reclamation when old states are pinned.

8 RELATED WORK

Dynamic graph storage. Static graph frameworks such as Ligra [36] and Ligra+ [37] provide CSR-like locality for traversal-heavy analytics, but are not designed for fine-grained online updates. Array-oriented dynamic designs, including LLAMA [24], CSR++ [15], and PPCSR [43], extend compact adjacency layouts with snapshot layers, buffers, or PMA-style gaps. Other systems use mutable or adaptive adjacency structures: STINGER [12] stores per-vertex edge blocks, GraphOne [21] separates ingest-friendly updates from analysis-friendly views, Aspen [10] supports lightweight snapshots with compressed functional trees, and Terrace [29] adapts containers to degree skew. Recent systems such as Spruce [35] and RisGraph [13] further optimize memory usage, indexing, and real-time updates. These designs improve update support, but are mainly optimized for dynamic graph processing rather than concurrent analytics with many snapshot readers.

Transactional graph stores. Transactional graph stores extend dynamic graph storage with isolation and atomic updates. Coupled designs such as LiveGraph [47] and GTX [46] place current edges and historical deltas in shared log- or delta-block structures, favoring high-throughput updates but requiring readers to filter invisible versions. Decoupled designs such as Teseo [8], Sortledton [16], and RapidStore [19] separate current adjacency from version management. Teseo and Sortledton preserve ordered adjacency for scans and intersections, while RapidStore uses subgraph-level copy-on-write snapshots to avoid per-edge version checks. These systems reduce different sources of read-write interference, but still face trade-offs among update granularity, scan locality, snapshot freshness, and version maintenance.

Graph databases. Graph databases provide a full DBMS layer over graph data, including property-graph or RDF models, declarative query languages, indexing, persistence, and transactions. Systems such as Neo4j [26], TigerGraph [9], and NebulaGraph [44] emphasize query expressiveness, distributed execution, and operational database features. Research systems such as GraphflowDB [25] and Kùzu [14] study columnar graph storage, list-based processing, and factorized execution, while benchmarks such as LDBC SNB [2] capture mixed graph query and update workloads. Compared with general-purpose graph databases, in-memory transactional graph storage focuses more narrowly on adjacency and version organization, where reducing MVCC checks, pointer chasing, and reader–writer interference is critical for traversal-heavy workloads.

9 CONCLUSION

This paper presents AVBGraph, an in-memory transactional graph storage engine for high-rate ingestion and concurrent traversal-heavy analytics. AVBGraph uses AVB to reorganize historical edge versions into contiguous Version Blocks grouped by invalidation epochs, replacing fine-grained MVCC chain probing with block-level pruning and scans. Its epoch-based batch commit and asynchronous publication protocol further decouples write ingestion from analytical visibility, while the dual-grained extension supports finer snapshot freshness when needed. Experiments on large dynamic graphs show that AVBGraph achieves high update throughput, keeps analytical performance close to static CSR baselines, and controls memory growth under mixed read-write workloads.

REFERENCES

[1] 2026. AVBGraph: Accumulated Version Blocks for Ingestion and Analytics in Transactional Graph Storage (Full Version). https://github.com/Mustingu/AVBGraph/blob/main/AVBGraph(fullversion).pdf

[2] Renzo Angles, János Benjamin Antal, Alex Averbuch, Altan Birler, Peter Boncz, Márton Búr, Orri Erling, Andrey Gubichev, Vlad Haprian, Moritz Kaufmann, et al. 2020. The LDBC social network benchmark. *arXiv preprint arXiv:2001.02299* (2020).

[3] Scott Beamer, Aydın Buluç, Krste Asanović, and David A. Patterson. 2013. *Distributed Memory Breadth-First Search Revisited: Enabling Bottom-Up Search.* Technical Report UCB/EECS-2013-2. EECS Department, University of California, Berkeley. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2013/EECS-2013-2.html

[4] Lawrence Benson, Leon Papke, and Tilmann Rabl. 2022. PerMA-bench: benchmarking persistent memory access. (2022).

[5] Maciej Besta, Marc Fischer, Vasiliki Kalavri, Michael Kapralov, and Torsten Hoefler. 2021. Practice of streaming processing of dynamic graphs: Concepts, models, and systems. *IEEE Transactions on Parallel and Distributed Systems* 34, 6 (2021), 1860–1876.

[6] Johannes Blum, Stefan Funke, and Sabine Storandt. 2021. Sublinear search spaces for shortest path planning in grid and road networks. *Journal of Combinatorial Optimization* 42, 2 (2021), 231–257.

[7] Chandra Chekuri, Kent Quanrud, and Manuel R Torres. 2022. Densest subgraph: Supermodularity, iterative peeling, and flow. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1531–1555.

[8] Dean De Leo and Peter Boncz. 2021. Teseo and the analysis of structural dynamic graphs. *Proceedings of the VLDB Endowment* 14, 6 (2021), 1053–1066.

[9] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor Lee. 2019. Tigergraph: A native MPP graph database. *arXiv preprint arXiv:1901.08248* (2019).

[10] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. 2019. Low-latency graph streaming using compressed purely-functional trees. In *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*. 918–934.

[11] Guanghan Duan, Hongwu Lv, Huiqiang Wang, Guangsheng Feng, and Xiaoli Li. 2024. Practical cyber attack detection with continuous temporal graph in dynamic network system. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4851–4864.

[12] David Ediger, Rob McColl, Jason Riedy, and David A Bader. 2012. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing*. IEEE, 1–5.

[13] Guanyu Feng, Zixuan Ma, Daixuan Li, Shengqi Chen, Xiaowei Zhu, Wentao Han, and Wenguang Chen. 2021. Risgraph: A real-time streaming system for evolving graphs to support sub-millisecond per-update analysis at millions ops/s. In *Proceedings of the 2021 International Conference on Management of Data*. 513–527.

[14] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoğlu. 2023. Kùzu graph database management system. In *The Conference on Innovative Data Systems Research*, Vol. 7. 25–35.

[15] Soukaina Firmli, Vasileios Trigonakis, Jean-Pierre Lozi, Iraklis Psaroudakis, Alexander Weld, Dalila Chiadmi, Sungpack Hong, and Hassan Chafi. 2020. Csr++: A fast, scalable, update-friendly graph data structure. In *24th International Conference on Principles of Distributed Systems (OPODIS'20)*.

[16] Per Fuchs, Domagoj Margan, and Jana Giceva. 2023. Sortledton: A universal graph data structure. *ACM SIGMOD Record* 52, 1 (2023), 17–25.

[17] Pankaj Gupta, Venu Satuluri, Ajeet Grewal, Siva Gurumurthy, Volodymyr Zhabiuk, Quannan Li, and Jimmy Lin. 2014. Real-Time Twitter Recommendation: Online Motif Detection in Large Dynamic Graphs. *Proc. VLDB Endow.* 7, 13 (2014), 1379–1380.

[18] Minyang Han and Khuzaima Daudjee. 2015. Giraph unchained: Barrierless asynchronous parallel execution in pregel-like graph processing systems. *Proceedings of the VLDB Endowment* 8, 9 (2015), 950–961.

[19] Chiyu Hao, Jixian Su, Shixuan Sun, Hao Zhang, Sen Gao, Jianwen Zhao, Chenyi Zhang, Jieru Zhao, Chen Chen, and Minyi Guo. 2025. RapidStore: An Efficient Dynamic Graph Storage System for Concurrent Queries. *Proc. VLDB Endow.* 18, 10 (2025), 3587–3600.

[20] Jiaxin Jiang, Yuhang Chen, Bingsheng He, Min Chen, and Jia Chen. 2024. Spade+: A generic real-time fraud detection framework on dynamic graphs. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 7058–7073.

[21] Pradeep Kumar and H Howie Huang. 2020. Graphone: A data store for real-time analytics on evolving graphs. *ACM Transactions on Storage (TOS)* 15, 4 (2020), 1–40.

[22] Hao Lin, Guannan Liu, Junjie Wu, Yuan Zuo, Xin Wan, and Hong Li. 2019. Fraud detection in dynamic interaction network. *IEEE Transactions on Knowledge and Data Engineering* 32, 10 (2019), 1936–1950.

[23] Mingqi Lv, Chengyu Dong, Tieming Chen, Tiantian Zhu, Qijie Song, and Yuan Fan. 2021. A heterogeneous graph learning model for cyber-attack detection. *arXiv preprint arXiv:2112.08986* (2021).

[24] Peter Macko, Virendra J Marathe, Daniel W Margo, and Margo I Seltzer. 2015. Llama: Efficient graph analytics using large multiversioned arrays. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 363–374.

[25] Amine Mhedhbi. 2023. GraphflowDB: Scalable Query Processing on Graph-Structured Relations. (2023).

[26] Neo4j, Inc. [n.d.]. *Neo4j Graph Database.*

[27] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast serializable multi-version concurrency control for main-memory database systems. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 677–689.

[28] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web.* Technical Report. Stanford infolab.

[29] Prashant Pandey, Brian Wheatman, Helen Xu, and Aydin Buluc. 2021. Terrace: A hierarchical graph container for skewed dynamic graphs. In *Proceedings of the 2021 international conference on management of data*. 1372–1385.

[30] Jiawei Ren, Deji Fu, Chunyu Shi, Zhilan Huang, Wanyi Zhu, and Yi Liu. 2023. Research on Cloud Computing Technology Graph Analysis. In *2023 2nd International Conference on Computing, Communication, Perception and Quantum Technology (CCPQT)*. IEEE, 84–91.

[31] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020).

[32] Yousef Saad. 2003. *Iterative methods for sparse linear systems.* SIAM.

[33] Siddhartha Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M Tamer Özsu. 2020. The ubiquity of large graphs and surprising challenges of graph processing: extended survey. *The VLDB journal* 29, 2 (2020), 595–618.

[34] Aneesh Sharma, Jerry Jiang, Praveen Bommannavar, Brian Larson, and Jimmy Lin. 2016. GraphJet: Real-time content recommendations at Twitter. *Proceedings of the VLDB Endowment* 9, 13 (2016), 1281–1292.

[35] Jifan Shi, Biao Wang, and Yun Xu. 2024. Spruce: a fast yet space-saving structure for dynamic graph storage. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.

[36] Julian Shun and Guy E Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*. 135–146.

[37] Julian Shun, Laxman Dhulipala, and Guy E Blelloch. 2015. Smaller and faster: Parallel processing of compressed graphs with Ligra+. In *2015 Data Compression Conference*. IEEE, 403–412.

[38] Weiping Song, Zhiping Xiao, Yifan Wang, Laurent Charlin, Ming Zhang, and Jian Tang. 2019. Session-based social recommendation via dynamic graph attention networks. In *Proceedings of the Twelfth ACM international conference on web search and data mining*. 555–563.

[39] Leyuan Wang, Yangzihao Wang, Carl Yang, and John D Owens. 2016. A comparative study on exact triangle counting algorithms on the gpu. In *Proceedings of the ACM Workshop on High Performance Graph Processing*. 1–8.

[40] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*. 2628–2638.

[41] Todd Warszawski and Peter Bailis. 2017. Acidrain: Concurrency-related attacks on database-backed web applications. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 5–20.

[42] Marcel Weisgut, Daniel Ritter, Pinar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3119–3133.

[43] Brian Wheatman and Helen Xu. 2021. A parallel packed memory array to store dynamic graphs. In *2021 Proceedings of the Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 31–45.

[44] Min Wu, Xinglu Yi, Hui Yu, Yu Liu, and Yujue Wang. 2022. Nebula Graph: An open source distributed graph database. *arXiv preprint arXiv:2206.07278* (2022).

[45] Zhen Zhang, Bingqiao Luo, Shengliang Lu, and Bingsheng He. 2023. Live graph lab: Towards open, dynamic and real transaction graphs with nft. *Advances in Neural Information Processing Systems* 36 (2023), 18769–18793.

[46] Libin Zhou, Lu Xing, Yeasir Rayhan, and Walid G Aref. 2025. GTX: A Write-Optimized Latch-free Graph Data System with Transactional Support. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.

[47] Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulnaga, and Wenguang Chen. 2019. LiveGraph: A transactional graph storage system with purely sequential adjacency list scans. *arXiv preprint arXiv:1910.05773* (2019).

A IMPLEMENTATION DETAILS

This appendix provides implementation details that are orthogonal to the main storage design but are useful for understanding how AVBGraph is realized in a dynamic transactional graph store.

A.1 Vertex Index

AVBGraph separates the vertex index from the edge-version storage. The vertex index maps sparse external vertex identifiers to dense internal identifiers, so that per-vertex metadata and adjacency pointers can be stored in compact, array-friendly structures. This design follows the common organization used by transactional graph stores: a concurrent hash map resolves external vertex IDs, while a dense metadata vector stores per-vertex descriptors.

Each vertex descriptor contains the entry points to the vertex-local storage structures, including the primary adjacency, the transient version blocks associated with unpublished epochs, the published VB chain or its head metadata, the vertex-local synchronization object, and lightweight degree or status counters used by traversal and update paths. Since traversal kernels operate on dense vertex IDs after lookup, the dense descriptor array provides predictable locality and avoids repeated hash-table access inside scan-heavy analytics.

To support dynamic growth, the dense metadata vector is implemented as a chunked or two-level vector. New dense IDs are allocated through an atomic ID allocator, and new chunks are created on demand. This avoids relocating existing descriptors when the vertex space grows, so edge-storage pointers and per-vertex metadata references remain stable. If a deployment supports vertex deletion, reusable dense IDs can be maintained through a lock-free free list; otherwise, edge-only workloads simply allocate dense IDs monotonically.

The vertex index is not the main optimization target of AVBGraph. Its role is to provide stable per-vertex entry points and to decouple sparse external IDs from the storage layout. For baselines that assume a dense ID domain, we use the same outer sparse-to-dense mapping wrapper when necessary, without modifying their original edge-storage designs. Therefore, the comparison focuses on the edge-version organization rather than on vertex-ID normalization.

A.2 Epoch Management Details

This appendix supplements the epoch-management mechanism in Section 4.2 with the derivation behind sampled sealing and the implementation details of asynchronous publication.

Sampled sealing. A direct implementation could maintain an exact per-epoch update counter and seal the epoch once the counter reaches a target size N. However, this would make every update contend on the same shared counter. AVBGraph instead uses sampled sealing. Each epoch maintains a small sampling quota h, where $h \ll N$, and each individual update generates a sealing hit with probability

$$p = \frac{h}{N}.$$

For a transaction containing n edge updates, the number of sampled hits is

$$X_n \sim \text{Binomial}(n, p).$$

The transaction adds X_n to the epoch's sampled-hit counter, and only when the counter reaches h does a worker attempt to seal the epoch. Thus, the shared counter is touched only by sampled hits rather than by every update.

Let U be the number of individual updates needed to obtain h sampled hits, ignoring the bounded overshoot of the final sealing transaction. Then U follows a negative-binomial distribution:

$$\Pr[U = u] = \binom{u-1}{h-1} p^h (1-p)^{u-h}, \qquad u \geq h.$$

Therefore,

$$\mathbb{E}[U] = \frac{h}{p} = N,$$

and

$$\text{Var}(U) = \frac{h(1-p)}{p^2} = \frac{h(1-h/N)}{(h/N)^2} = \frac{N(N-h)}{h}.$$

Hence, sampled sealing preserves the target expected epoch size N, while the sampling quota h controls concentration. In particular, the relative standard deviation is

$$\frac{\sqrt{\text{Var}(U)}}{\mathbb{E}[U]} = \sqrt{\frac{N-h}{hN}} \approx \frac{1}{\sqrt{h}} \quad \text{when } N \gg h.$$

Thus, increasing h makes the realized epoch size more stable, but the expected number of sampled hits per epoch remains only h.

Concentration. The concentration of U can be related to standard Chernoff bounds by viewing sampled hits in a fixed prefix of updates. Let $Y_m \sim \text{Binomial}(m, p)$ be the number of hits among the first m updates. For $0 < \epsilon < 1$, the event $U < (1-\epsilon)N$ implies that at least h hits occur within $(1-\epsilon)N$ updates, while the event $U > (1+\epsilon)N$ implies that fewer than h hits occur within $(1+\epsilon)N$ updates. Applying Chernoff bounds gives

$$\Pr[U < (1-\epsilon)N] \leq \exp\left(-\frac{h\epsilon^2}{2-\epsilon}\right),$$

and

$$\Pr[U > (1+\epsilon)N] \leq \exp\left(-\frac{h\epsilon^2}{2(1+\epsilon)}\right).$$

Therefore,

$$\Pr[(1-\epsilon)N \leq U \leq (1+\epsilon)N] \geq 1 - \exp\left(-\frac{h\epsilon^2}{2-\epsilon}\right) - \exp\left(-\frac{h\epsilon^2}{2(1+\epsilon)}\right).$$

This bound is conservative. For example, when $h = 32$, the exact negative-binomial distribution gives $\Pr[0.5N \leq U \leq 1.5N] \approx 99.4\%$ for large N, showing that a small sampling quota already keeps epoch sizes close to the target while avoiding a per-update global counter.

Concurrent sealing. When a transaction observes that the sampled-hit counter has reached the quota h, it acquires a lightweight sealing lock and rechecks whether gwe still points to the same open epoch. If another worker has already advanced gwe, the attempt is aborted. Otherwise, the worker seals the current epoch, creates a new open epoch, resets the sampled-hit counter, advances gwe, and releases the lock. This lock does not serialize ordinary epoch admission: most transactions produce no sampled hit, and even sampled-hit updates only contend on the sampled counter. The sealing lock is acquired only when the quota is crossed, so it stays off the common write path.

Slot-based commit detection and ordered publication. After an epoch is sealed, AVBGraph must determine when all writers assigned to that epoch have finished. Maintaining a per-epoch active-transaction counter would again introduce centralized write-path synchronization. Instead, each worker records its current epoch and access mode in a local activity slot. The publisher periodically scans these slots to determine whether the next sealed epoch after gre is still referenced by any active writer.

To avoid missing a writer that enters an epoch concurrently with sealing, a worker registers through a lightweight guard: it first reads the current gwe, writes that epoch into its activity slot, and then rereads gwe. If gwe has changed, the worker retries registration. This read-gwe, publish-slot, reread-gwe sequence ensures that the publisher cannot skip a writer that joined an epoch during the scan.

The publisher always processes epochs in increasing order. Once the next epoch after gre is sealed and no active writer slot references it, the publisher marks it committed, transforms its TVB records into per-vertex VBs, links those VBs into the corresponding vertex histories under vertex-local locks, and advances gre only after all affected vertices have been processed. Later epochs cannot bypass an earlier unpublished epoch, even if they become committed earlier. This ordered publication rule preserves a monotone reader-visible prefix and prevents readers from observing a partially published epoch.

B SNAPSHOT-VISIBILITY CORRECTNESS

This appendix gives the snapshot-visibility argument for the two visibility modes of AVBGraph. The goal is to show that readers observe only committed versions, never observe a partially published epoch, and see all updates of a transaction at one visibility boundary.

B.1 Safety Invariants

We first state the invariants shared by both modes.

Invariant 1: Immutable published VBs. Once a VB is linked into the per-vertex VB chain, it is never modified in place. Later updates may create newer VBs and link them ahead of the existing chain, but the contents of an already published VB remain stable.

Invariant 2: Vertex-local structural consistency. For each vertex, the primary adjacency, TVB state, and VB-chain head are modified only while holding the vertex-local write lock. A reader acquires the read lock to capture the primary-adjacency handle and the VB-chain head, and then traverses immutable published VBs without further synchronization.

Invariant 3: Per-edge lineage preservation. When an update replaces the latest version of an edge, the displaced version is recorded as historical state with the corresponding invalidation timestamp. Thus, for each edge, the primary record and its historical records still form a logical MVCC lineage, even though AVB physically groups historical records by invalidation epoch inside VBs.

Invariant 4: Commit-state consistency. Records from aborted transactions are removed or ignored before publication, and uncommitted records are never exposed as visible historical versions. In coarse-grained mode, the publisher materializes only committed records. In dual-grained mode, readers access unpublished TVB records only under vertex-local protection, and only committed records can be observed.

Invariant 5: Ordered publication. The global read epoch gre advances only after all VBs of the next epoch have been linked into the corresponding vertex histories. Publication is performed in epoch order, so a later epoch cannot become reader-visible before an earlier epoch.

Invariant 6: Atomic transaction boundary. All updates of the same transaction share one reader-visible boundary. In coarse-grained mode, this boundary is the transaction's epoch. In dual-grained mode, it is the transaction's composite commit timestamp. Therefore, a reader cannot observe only a subset of a transaction due to different timestamps on different vertices.

Invariant 7: Fixed reader snapshot. A reader fixes its snapshot boundary at the beginning of the scan and applies the same visibility rule throughout the traversal. Later epoch advancement or publication does not change the snapshot boundary of an already running reader.

B.2 Coarse-Grained Mode

In coarse-grained mode, a reader observes the graph at an epoch boundary. Let e_r be the reader's snapshot epoch. The reader consults the primary adjacency and published VBs, but does not inspect TVBs from open, sealed, or committed-but-unpublished epochs.

No uncommitted records. A sealed epoch becomes committed only after all writers assigned to that epoch have completed. During publication, aborted records are ignored and only committed updates are transformed into VBs. Since coarse-grained readers do not inspect unpublished TVBs, they cannot observe records from active or aborted transactions.

No partially published epoch. The publisher advances gre only after the entire epoch has been published for all affected vertices. Therefore, if a reader chooses $e_r \leq \text{gre}$, every epoch included in its snapshot has already been fully linked into the relevant vertex histories. If an epoch has not been fully linked, gre has not advanced past it, and that epoch cannot be selected as visible by a coarse-grained reader.

Atomic visibility of transactions. All transactions admitted to the same epoch share the same coarse-grained visibility boundary. Hence, for a reader at epoch e_r, an epoch is either included in the published prefix or excluded from it. A transaction cannot be partially visible across vertices because publication does not expose the epoch until all per-vertex VBs of that epoch have been linked.

Snapshot consistency. For each edge, AVB preserves the same logical version intervals as conventional MVCC: a version record τ is visible to snapshot e_r iff $c(\tau) < e_r \leq \text{inv}(\tau)$. AVB changes only the physical placement of historical records, grouping them into VBs by invalidation epoch. Because published VBs are immutable and the reader uses a fixed e_r, the reader observes a consistent epoch-level snapshot.

Therefore, coarse-grained mode provides snapshot isolation at epoch granularity. Transactions in the same epoch intentionally share one reader-visible commit boundary, which trades per-transaction freshness for batched publication and low write-path coordination.

B.3 Dual-Grained Mode

Dual-grained mode refines epoch-level visibility with an intra-epoch commit order. A reader takes a composite snapshot timestamp $T_r = (e_r, t_r)$, and timestamps are compared lexicographically. A version record τ is visible iff

$$c(\tau) < T_r \leq \mathrm{inv}(\tau).$$

No uncommitted records. In dual-grained mode, a writer holds the write locks of all touched vertices until the transaction commits or aborts. Before commit, readers cannot acquire the corresponding read locks and therefore cannot observe the transaction's uncommitted primary-adjacency or TVB state. If the transaction aborts, the writer still holds the affected locks and rolls back the primary adjacency using the undo information in the TVB, then removes or invalidates its TVB records. Thus, aborted or uncommitted records are not visible to readers.

Atomic transaction visibility. At commit time, a transaction obtains a single composite commit timestamp. All records produced by the transaction use this same timestamp boundary. The writer releases the touched vertex locks only after all updates of the transaction have been installed or marked committed. Therefore, any reader that starts after the commit boundary can observe all relevant records according to the timestamp rule, while a reader whose snapshot precedes the commit boundary observes none of them.

Correctness over published VBs. For published VBs, dual-grained readers use the same block-oriented access path as coarse-grained readers, with additional intra-epoch checks only when needed. If a block's epoch is older than the reader's active epoch, the block is outside the visible interval and can be skipped. If it is newer, the reader can scan in creation-timestamp order and stop once records are no longer earlier than T_r. If the block belongs to the reader's active epoch, the reader checks the full condition $c(\tau) < T_r \leq \mathrm{inv}(\tau)$, because records in the same epoch may differ in intra-epoch commit order. These checks are exactly the MVCC visibility rule over composite timestamps.

Correctness over unpublished TVBs. An unpublished TVB is not yet reorganized into a creation-ordered immutable VB, so it does not provide the same early-stopping property as a published VB. When a dual-grained reader needs to inspect such a TVB, it does so while holding the vertex-local read lock. This excludes concurrent writers that may be committing or aborting on the same vertex. The reader then tests each committed TVB record using the full composite timestamp condition. Hence, unsorted TVB layout affects only scan cost, not visibility correctness.

Snapshot consistency across vertices. A reader fixes T_r before traversal and applies the same timestamp rule at every vertex. Although different vertices may be read at different physical times, the logical visibility decision depends only on T_r, the record creation timestamp, and the record invalidation timestamp. Since all updates of one transaction share the same composite timestamp, the reader cannot observe a transaction on one vertex but miss it on another due to inconsistent timestamp boundaries.

Therefore, dual-grained mode provides transaction-level snapshot visibility under vertex-local protection. Compared with the coarse-grained mode, it exposes committed updates within the current epoch earlier, while preserving AVB's immutable published VBs, ordered publication, and block-oriented historical layout.

C SAFE RECLAMATION

This appendix describes why obsolete published VBs can be reclaimed safely. The key safety requirement is stronger than visibility alone: a reclaimed record must be neither visible to nor reachable by any active reader. This is important because AVBGraph supports both block-oriented historical scans and, when ordered access is required, predecessor-reference traversal from the primary adjacency.

AVBGraph uses delayed, epoch-based reclamation. Each reader publishes its snapshot boundary before it captures any vertex-local entry point, and clears its reader state after the scan finishes. Let $(\mathcal{R})$ denote the set of active readers, and let (T_r) be the snapshot boundary of reader (r). The reclamation frontier is defined as the oldest active reader boundary:

$$T_{\min} = \min_{r \in \mathcal{R}} T_r.$$

If there is no active reader, $(T_{\min})$ is treated as the latest published boundary. For a published VB (B), let $(\mathrm{ub}(B))$ denote the largest invalidation boundary of records contained in (B). Since VBs are organized by invalidation epochs, this value is simply the block invalidation epoch in coarse-grained mode, and the corresponding composite upper bound in dual-grained mode.

A published VB is eligible for retirement only if

$$\mathrm{ub}(B) < T_{\min}.$$

This condition means that every record in (B) has been invalidated strictly before the oldest active snapshot. Therefore, no active reader can observe any record in (B) as visible under the MVCC rule

$$c(\tau) < T_r \leq \mathrm{inv}(\tau).$$

If $(\mathrm{inv}(\tau) < T_{\min} \leq T_r)$, then $(T_r \leq \mathrm{inv}(\tau))$ is false for every active reader.

Retirement and physical freeing are separated. Once a VB satisfies the frontier condition, AVBGraph unlinks it from the corresponding vertex history under the vertex-local lock and places it into a retire list. The memory of the retired VB is freed only after a grace period, i.e., after all readers that were active at the time of retirement have exited. This prevents a reader that has already captured an old VB-chain head from dereferencing freed memory.

This two-step rule also covers ordered scans that use predecessor references. Consider an active reader with snapshot (T_r). If a historical record (τ) has $(\mathrm{inv}(\tau) < T_r)$, then (τ) is already obsolete for that reader. Moreover, following predecessor references beyond such a record cannot lead to a visible version for (T_r), because older predecessors have even earlier validity intervals. Hence, a block whose upper invalidation boundary is smaller than (T_r) is not needed as a visible target or as a necessary intermediate record in predecessor traversal. If a reader may still hold a raw pointer to that block because it captured the chain before unlinking, the grace period delays physical freeing until that reader finishes.

Therefore, after the frontier condition and the grace period both hold, no active reader can either observe or safely reach the reclaimed VB. Future readers cannot reach it because it has already been unlinked from the published history, and existing readers have quiesced before the memory is released.

The same argument applies independently to each vertex history because published VBs are immutable after linking, and structural

changes to the primary adjacency, TVB state, and VB-chain head are protected by the vertex-local lock. Transient TVB storage is reclaimed separately as part of the epoch publication lifecycle: unpublished records are protected by vertex-local locks while they may be inspected by dual-grained readers, and only committed records are materialized into immutable VBs. After publication, obsolete published history is reclaimed by the block-oriented rule above.

Compared with traditional per-edge version chains, this policy allows AVBGraph to reclaim history at block granularity. Since many obsolete records with nearby invalidation boundaries are accumulated into the same VB, reclamation can retire and free contiguous historical regions instead of chasing and unlinking individual edge-version nodes.

D OPTIONAL TRANSACTIONAL EXTENSIONS

This appendix discusses two optional extensions of AVBGraph: read-write transactions and serializable execution. These extensions reuse the same vertex-local protection mechanism as the main protocol and are not the focus of our evaluation.

D.1 Read-Write Transactions

Read-write transactions can be supported most naturally in the dual-grained mode. A transaction fixes a snapshot timestamp at start time for ordinary snapshot reads. For vertices that it updates, the transaction retains the corresponding vertex-local write locks until commit or abort. Its newly written records are kept either in its private write set or in TVB records tagged with the transaction state, so later reads in the same transaction first consult this private overlay before falling back to the committed snapshot state. This provides read-own-writes semantics without exposing uncommitted records to other readers.

At commit time, the transaction obtains a single composite commit timestamp, and all of its updates are assigned this same timestamp boundary. The updates then follow the normal dual-grained visibility path: the newest values are installed in the primary adjacency, the displaced historical records remain in the TVB, and the publisher later materializes committed TVB records into immutable VBs. If the transaction aborts, it still holds the touched vertex-local locks and can roll back the primary adjacency using the undo information recorded in the TVB or private write set.

This extension preserves snapshot isolation because public readers still apply the same timestamp visibility rule and never observe uncommitted records. The only additional rule is local to the running transaction: its own reads are evaluated over the committed snapshot plus its private updates.

D.2 Serializable Extension

A serializable variant can be obtained by retaining read locks as well as write locks until transaction end, yielding a strict two-phase-locking style protocol over vertex-local objects. A transaction acquires read locks for vertices it inspects and write locks for vertices it modifies; locks are released only after commit or abort. The existing no-wait retry policy can be used to avoid deadlock: if a transaction fails to acquire a required lock, it releases the locks acquired in the current attempt, backs off, and retries or aborts.

This extension is mainly suitable for short OLTP-style graph transactions with small read and write sets. It is not the default mode for long-running graph analytics, because analytical traversals may touch a large fraction of the graph and would hold many read locks for a long time, substantially increasing reader-writer interference and reducing the benefit of AVBGraph's snapshot-oriented design. Therefore, our evaluation focuses on the snapshot-isolated dual-grained and coarse-grained modes.

E DETAILED DERIVATION OF THE COST MODEL

This appendix provides the derivation details omitted from Section 6 and explains the refinements used in the AVB cost formula.

E.1 Expected Number of Affected Edges and Candidate VBs

Consider a snapshot scan of a vertex v with out-degree d. We assume that updates to each edge follow an independent Poisson process with mean μ updates per edge per epoch. Over a lag window of Δ epochs, the number of updates received by one edge is therefore

$$X_e \sim \text{Poisson}(\mu\Delta).$$

An edge is affected during the lag window if it is updated at least once. Thus,

$$\Pr[X_e \geq 1] = 1 - \Pr[X_e = 0] = 1 - e^{-\mu\Delta}.$$

By linearity of expectation, the expected number of distinct affected edges among the d outgoing edges of v is

$$d_h = \sum_{e \in N(v)} \Pr[X_e \geq 1] = d(1 - e^{-\mu\Delta}).$$

At the VB level, we ask whether vertex v contributes any historical record to a given epoch. Since the aggregate update rate over the d outgoing edges is $d\mu$ per epoch, the number of updates to v in one epoch follows

$$Y_v \sim \text{Poisson}(d\mu).$$

Therefore, the probability that v has a non-empty VB in one epoch is

$$\Pr[Y_v \geq 1] = 1 - e^{-d\mu}.$$

Over Δ lagging epochs, the expected number of non-empty candidate VBs that a snapshot scan may need to inspect is

$$B_v(\Delta) = \Delta(1 - e^{-d\mu}).$$

E.2 N2O Chain Cost

In an N2O-style layout, visibility is resolved independently for each edge. After scanning the current adjacency, a reader may need to follow version-chain pointers to skip versions that are newer than its snapshot. For one edge, the expected number of such newer historical versions in the lag window is $\mu\Delta$. Across d edges, the expected number of chain probes is $d\mu\Delta$.

Thus, the expected chain-based visibility cost is

$$C_{\text{chain}}(v, \Delta) \approx dc_s + d\mu\Delta c_r.$$

Here, dc_s is the cost of scanning the current adjacency, and $d\mu\Delta c_r$ is the dominant random or pointer-chasing cost of resolving historical visibility through edge-level chains.

E.3 AVB Cost and Its Refinements

In AVB, historical versions are grouped into VBs. A snapshot scan still pays dc_s to scan the current adjacency, but historical visibility is resolved at block granularity. For each non-empty candidate VB, the reader pays one random block-entry cost, giving the term

$$B_v(\Delta)c_r.$$

After entering candidate VBs, the remaining historical records are scanned sequentially. A basic approximation would charge one sequential access for each useful historical record:

$$C_{\text{AVB}}^{\text{basic}}(v,\Delta) \approx dc_s + B_v(\Delta)c_r + d_h c_s.$$

The refined formula uses three accounting rules.

First, AVB pays random access at VB granularity rather than at individual edge-version granularity. Thus, the random term is $B_v(\Delta)c_r$ instead of $d\mu\Delta c_r$.

Second, the useful historical work is bounded by the number of distinct affected edges, d_h, because multiple updates to the same edge within the lag window still contribute at most one visible replacement record to a snapshot scan.

Third, block-entry accounting avoids charging the first useful touch in a candidate VB twice. If a candidate VB is pure, meaning that all records in the block are visible to the snapshot, the block-entry access already accounts for one useful touch. For a non-pure VB, the scan may additionally touch one invisible boundary record, which offsets this saving. Hence only pure VBs receive a one-touch credit. Moreover, the remaining sequential work cannot exceed the total number of historical records beyond the first record of each candidate VB, namely $d\mu\Delta - B_v(\Delta)$.

Therefore, the effective sequential work is

$$x_{\text{seq}}(\Delta) = \min\left\{d_h - \min\{q_p B_v(\Delta), d_h\}, d\mu\Delta - B_v(\Delta)\right\}.$$

E.4 Expected Speedup

Combining the two costs, the expected speedup of AVB over N2O-style chain-based visibility resolution is

$$S(\Delta) = \frac{C_{\text{chain}}(v,\Delta)}{C_{\text{AVB}}(v,\Delta)} \approx \frac{dc_s + d\mu\Delta c_r}{dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s}.$$

This expression captures the main trend: N2O pays visibility-resolution cost through many edge-level random probes, while AVB replaces most of these probes with block-entry checks and sequential in-block scans. The benefit therefore increases when the lag window Δ, the degree d, the random-access penalty c_r/c_s, or the pure-block fraction q_p increases.

F COMPONENT STUDY AND BASELINE DETAILS

This appendix supplements the evaluation with (i) a component study isolating two epoch-management design choices, and (ii) the detailed modifications applied to RapidStore. All component-study experiments use the same concurrent setting as Section 7.3 (32 writer and 32 reader threads on Graph500-24, readers running PageRank), so each mechanism is measured under realistic read–write interference rather than in isolation. Results are shown in Figure 15.

F.1 Effect of Epoch Size N

We vary the sealing threshold N from 10^3 to 10^7. Figure 15(a) shows that ingestion throughput rises monotonically with N and saturates at large N: it grows from 1.78 to 5.49 MEPS for AVBGraph, and from 1.73 to 4.76 MEPS for AVBGraph-dual. A larger epoch lets more records share the same invalidation timestamp, producing larger Version Blocks that amortize visibility checks, publication, and reclamation over more records and that are scanned more sequentially; below $N \approx 10^6$ the epochs are too small to amortize this fixed per-epoch work, which is why throughput climbs steeply before flattening.

PageRank latency (Figure 15(b)) stays essentially flat across N (roughly 4–6 s, within run-to-run noise). This is expected: PageRank is a long scan-heavy analytic whose runtime dominates the epoch publication delay, so enlarging N does not slow it. The cost of a large N is instead a *semantic* one and is not a latency effect: under coarse-grained visibility an epoch is the unit of reader-visible publication, so a larger N makes the smallest visible commit boundary coarser—updates become visible only N edges at a time. This freshness coarsening, rather than any latency penalty, is what the dual-grained extension (Section 4.3) is designed to refine. Thus N trades batching efficiency against visibility granularity for analytics-dominated workloads, while the dual-grained mode decouples the two when finer freshness is required.

F.2 Effect of Sampled Sealing

We replace sampled sealing with an exact per-update counter that every update increments to detect the sealing threshold (the "×" markers at $N = 10^7$ in Figure 15). Removing sampled sealing reduces ingestion throughput from 5.49 to 4.87 MEPS for AVBGraph (−11.3%) and from 4.76 to 3.88 MEPS for AVBGraph-dual (−18.5%), while PageRank latency is essentially unchanged. Under 32 concurrent writers an exact global counter becomes a contention point on the common write path, whereas sampled sealing touches shared state only on a sampled hit and therefore keeps the write path lightweight. The larger drop for AVBGraph-dual is consistent with its write path already holding vertex-local locks until commit, so additional global contention compounds more sharply.

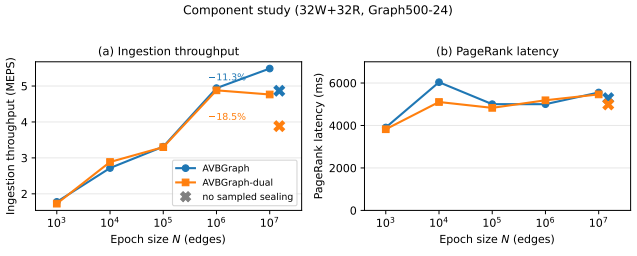

Figure 15: Component study under 32W+32R on Graph500-24. Throughput rises with N and saturates; PageRank latency is insensitive to N. The "×" markers replace sampled sealing with an exact per-update counter at $N = 10^7$.

F.3 RapidStore Modifications

RapidStore's public implementation is tuned for its own concurrent-query benchmark and exposes two issues outside that setting, both of which we had to address to run it under our mixed read–write workloads.

The first is a correctness issue. Reader registration is not protected when readers and writers run concurrently outside the original benchmark harness: a reader may register against a snapshot that a concurrent writer is simultaneously advancing, producing inconsistent or crashing runs. We guard reader registration with a lock so that all of our concurrent experiments can execute correctly. This guard is required for correctness and adds synchronization on the read path; it can therefore only *lower* RapidStore's read–write performance, never raise it.

The second is a stability issue we were unable to fix. Under concurrent updates—as opposed to RapidStore's read-mostly benchmark—its garbage-collection path is unstable, and we could not repair it within the original design. To obtain stable measurements we instead disable garbage collection and part of the version-generation work. Disabling reclamation and version maintenance removes work from the write path, so it can only *raise* RapidStore's write throughput and reduce its memory pressure; it does not model a deployable correct system, but it gives RapidStore an optimistic performance allowance.

Combining the two, the only change that can reduce RapidStore's performance is the correctness-required read guard, while the change we could not fix removes work and can only improve performance. The net effect is that our reported RapidStore numbers are an *optimistic upper bound* on what a fully correct implementation could achieve in this setting. Because disabling reclamation makes its memory footprint non-comparable, we omit RapidStore from the memory-footprint study (Section 7.4). Even against this upper bound, AVBGraph sustains higher update throughput and lower analytical latency across the concurrent experiments.